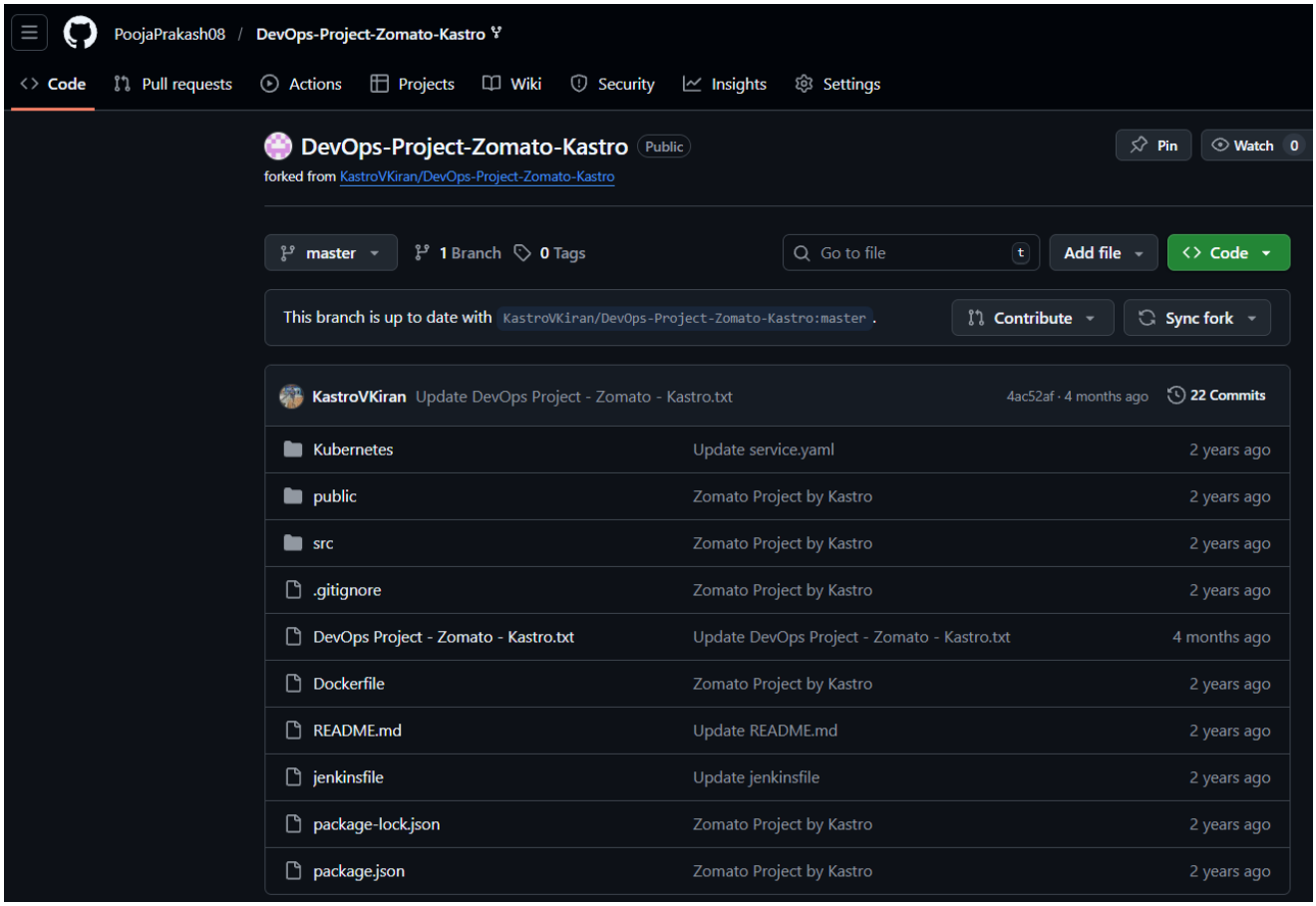
Deployment of Zomato Clone Application

End-to-End DevOps Deployment with CI/CD, Kubernetes and Monitoring

Name: Pooja Prakash Belgaumkar

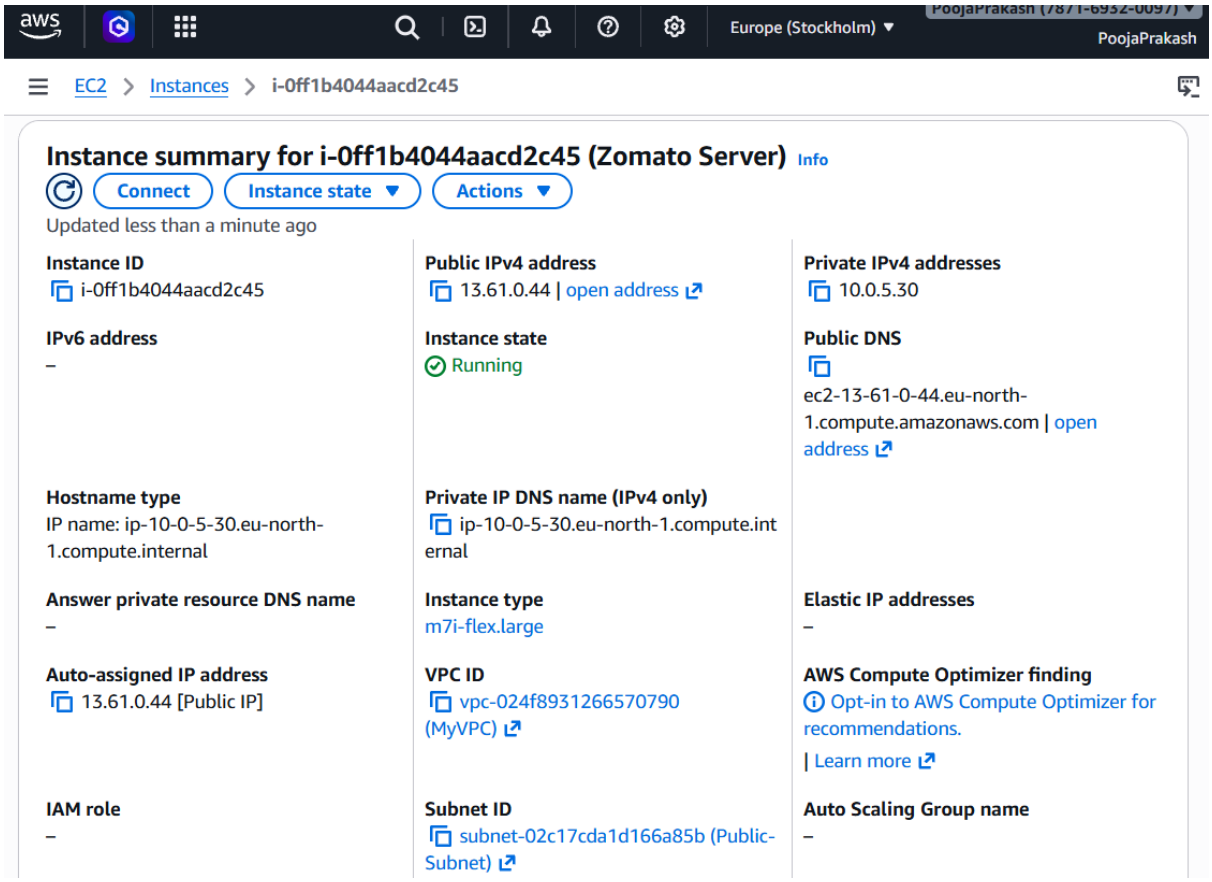
- This project demonstrates a complete DevOps lifecycle for deploying a Zomato web application. It covers containerization, CI/CD automation, Kubernetes orchestration, security scanning, email notifications and real-time monitoring, all implemented on AWS cloud infrastructure.



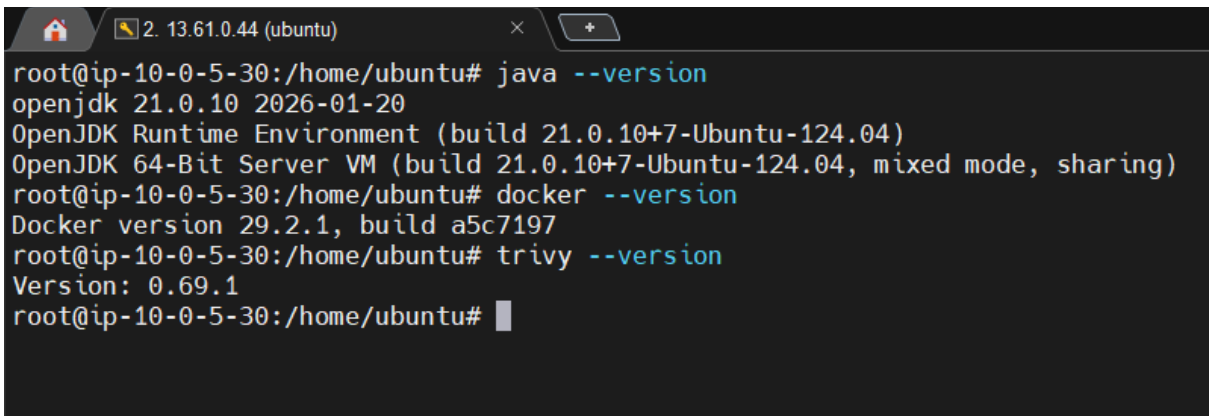
Phase 1: Docker Containerization and CI/CD Pipeline

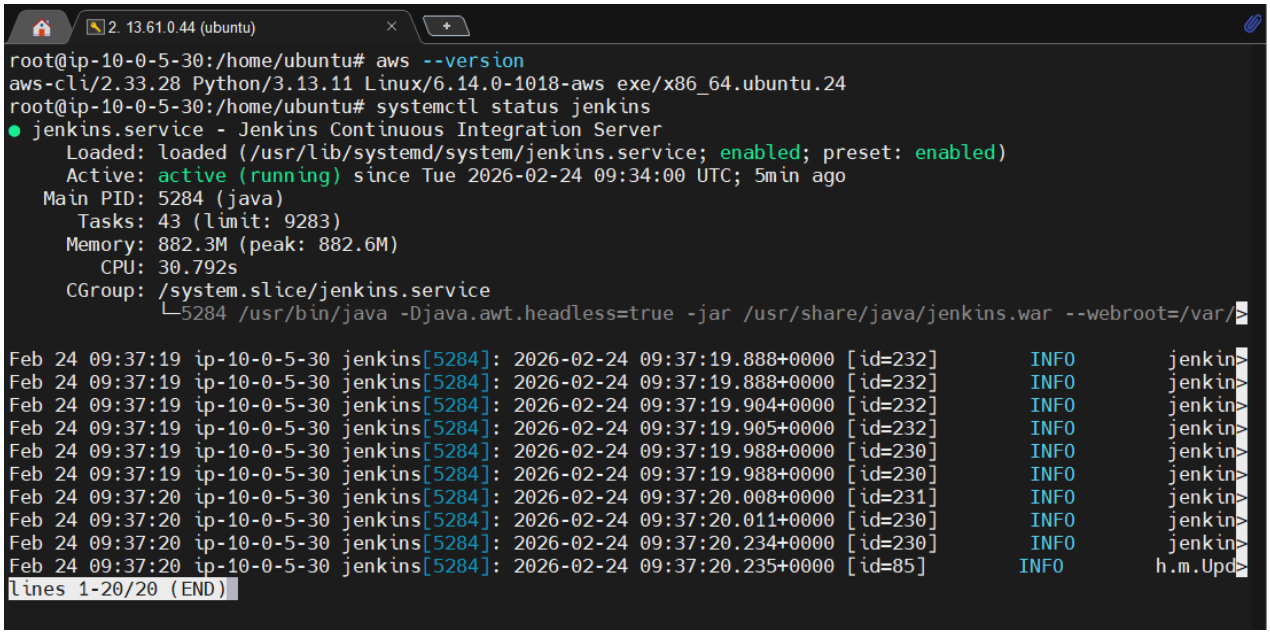
1. Infrastructure Setup

- Launched an Ubuntu EC2 instance named **Zomato Server** with a large instance type and 30 GB storage.

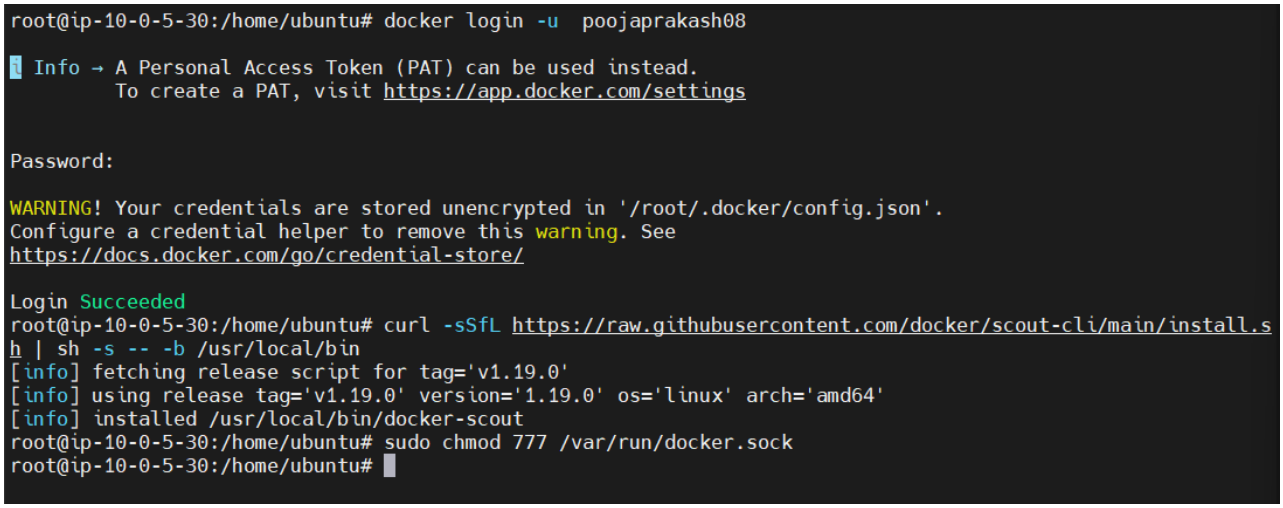


- Connect to the instance and Installed java, Docker, Trivy, AWS CLI and Jenkins

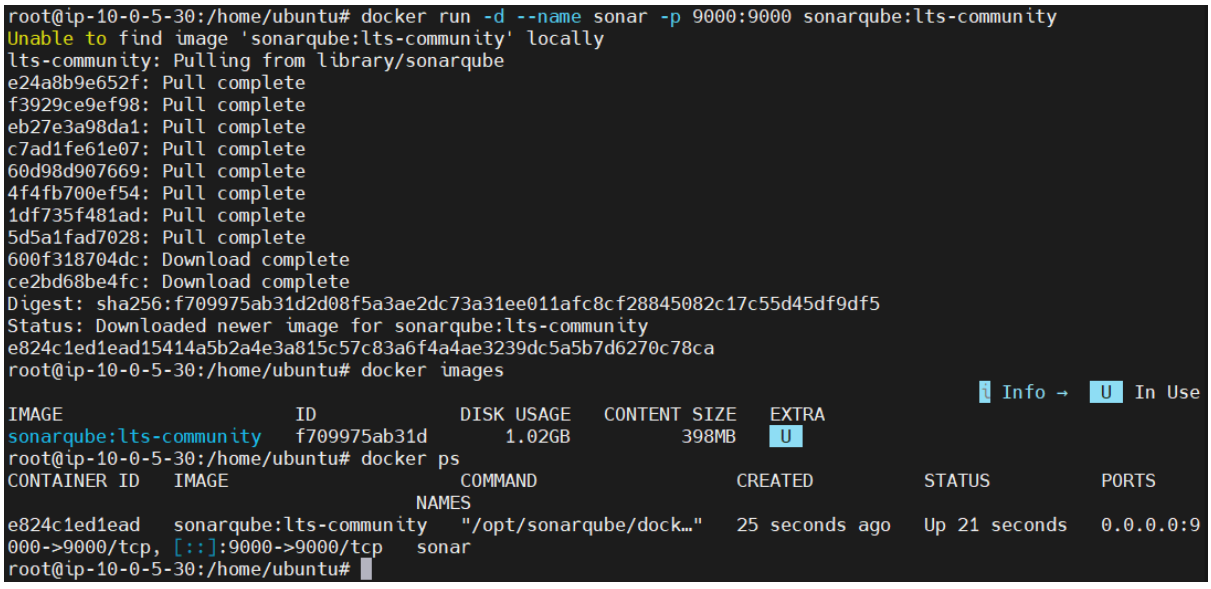




- Logged in to Docker Hub and installed Docker Scout.

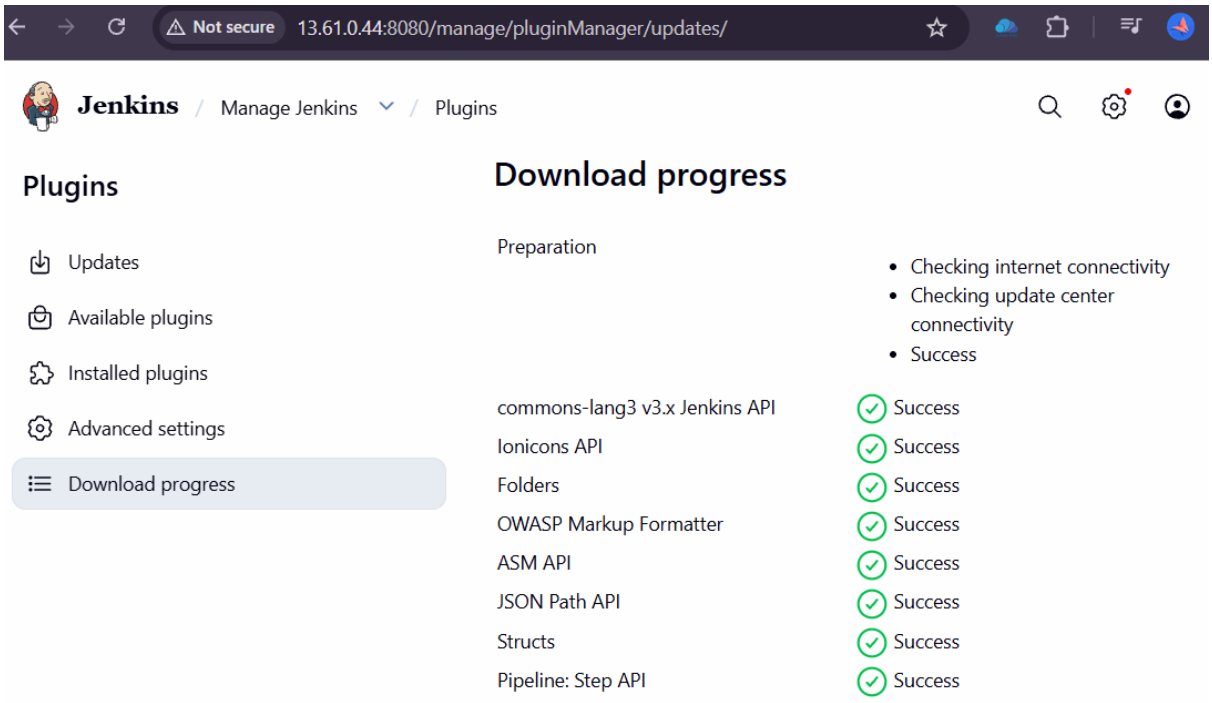


- Installed SonarQube using Docker.

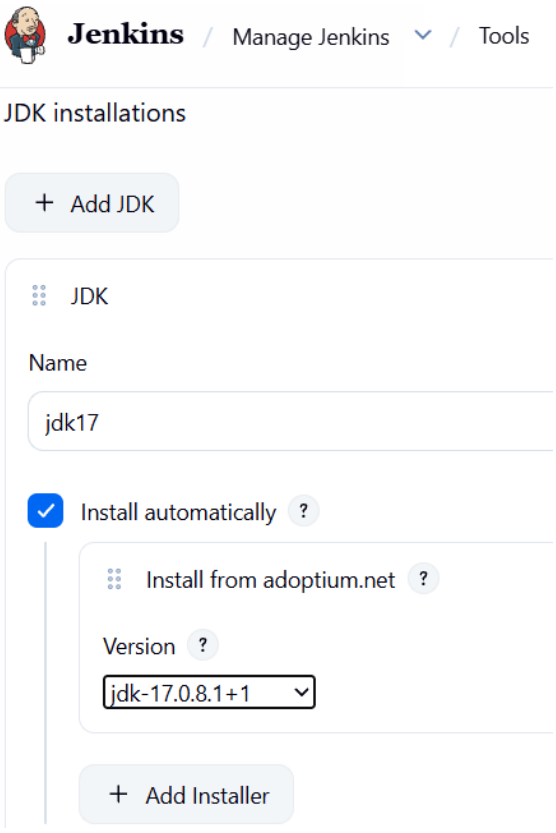


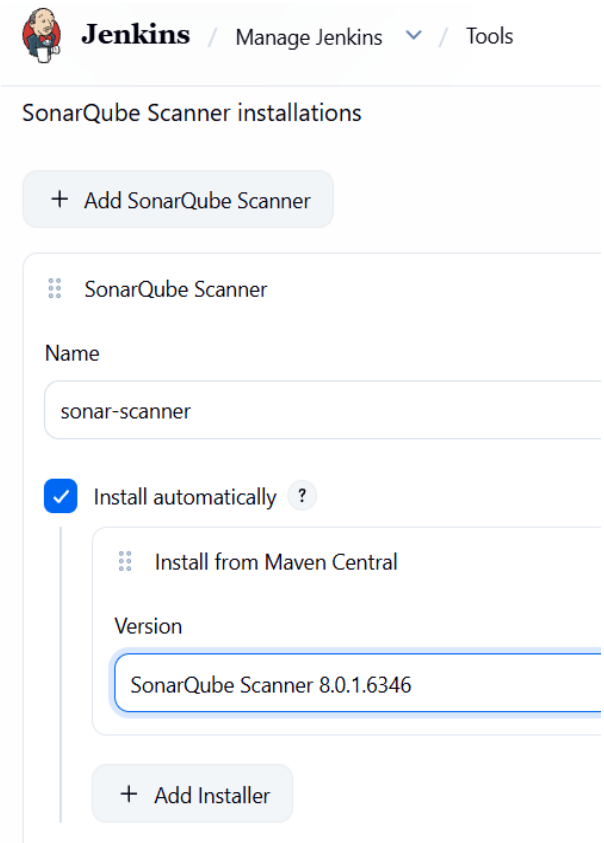
2. Jenkins Configuration

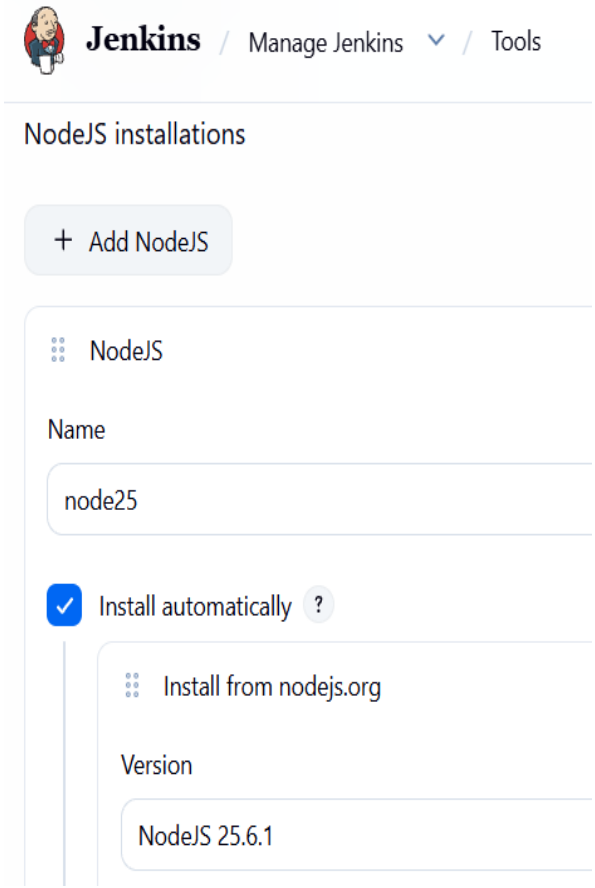
- Accessed the Jenkins web interface on port 8080 and installed required plugins.

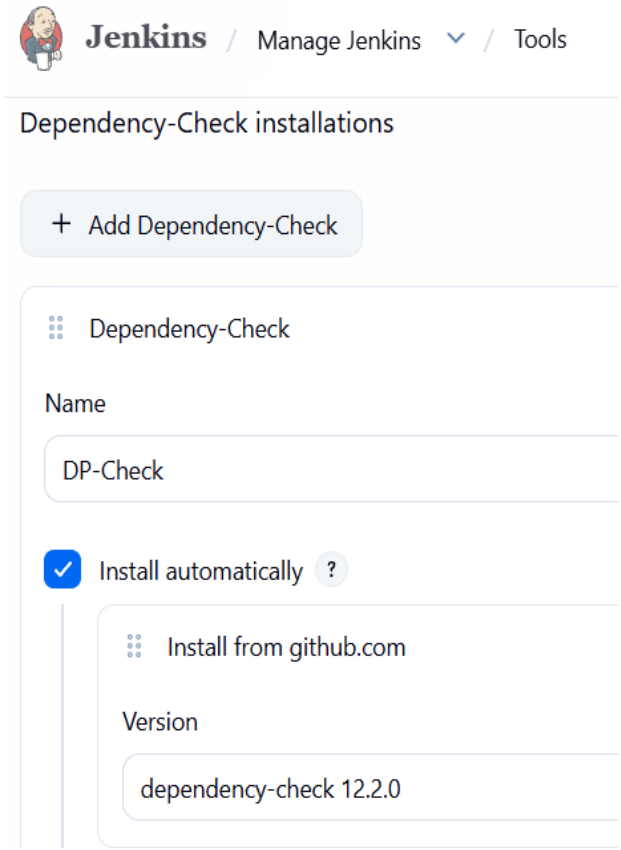


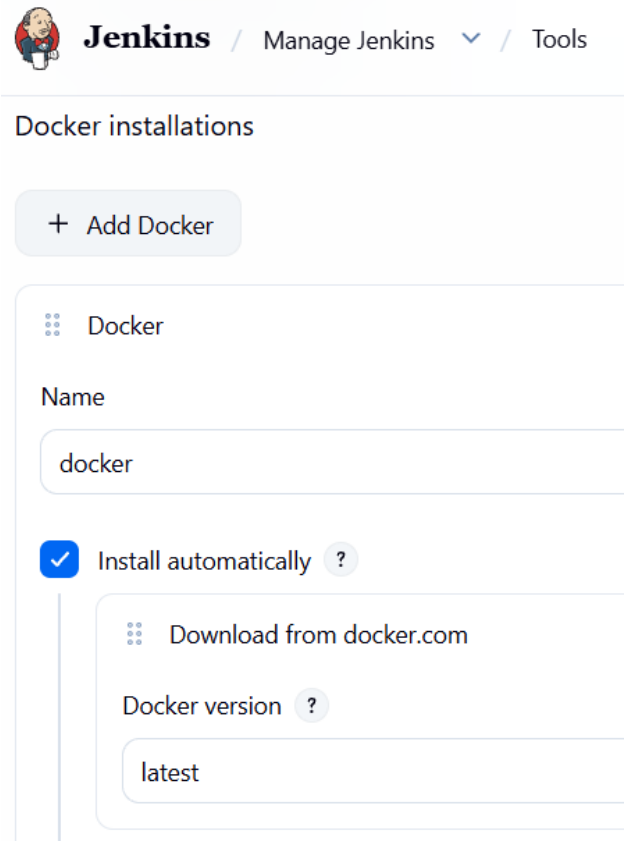
- Configured global tools in Jenkins: JDK, SonarQube, Nodejs, OWASP Dependency-Check, Docker tools in the Jenkins to use these tools in pipeline



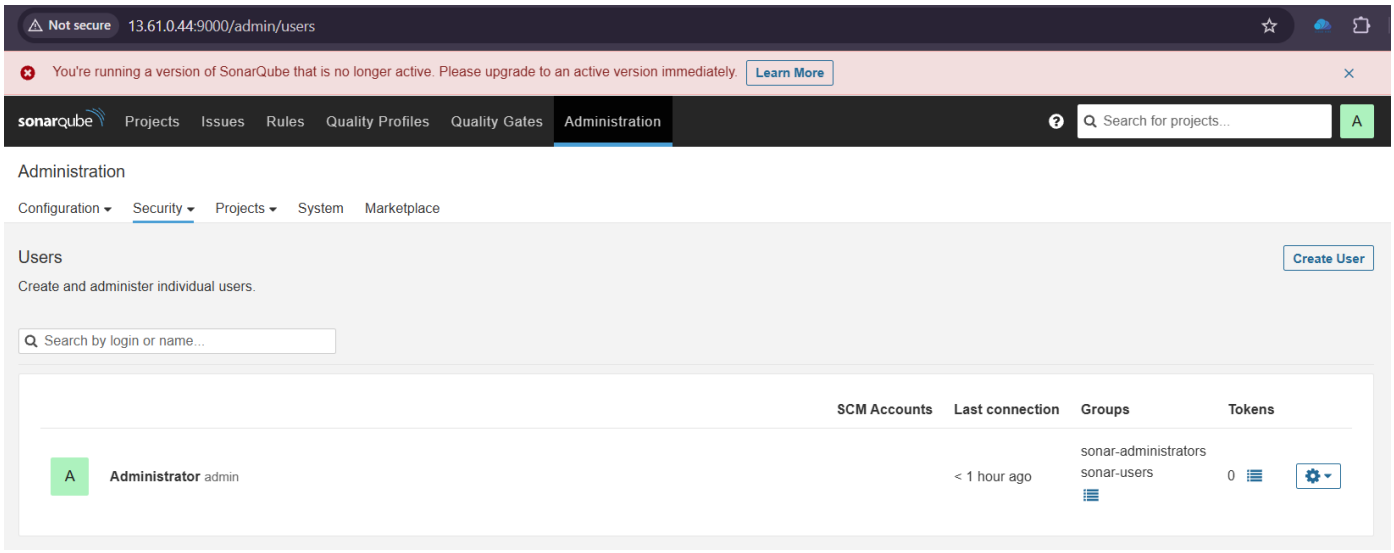


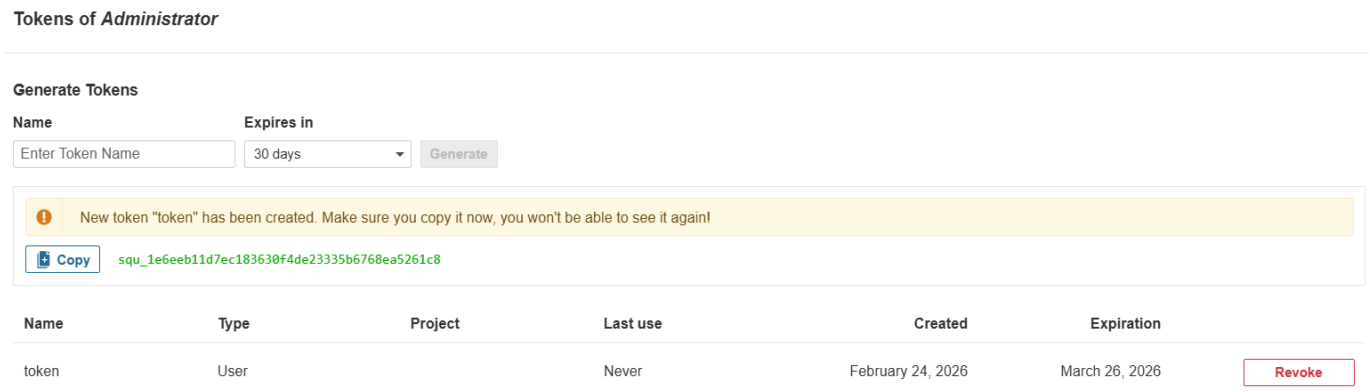

Jenkins / Manage Jenkins / Tools
NodeJS installations
+ Add NodeJS
NodeJS
Name
node25
Install automatically
Install from nodejs.org
Version
NodeJS 25.6.1

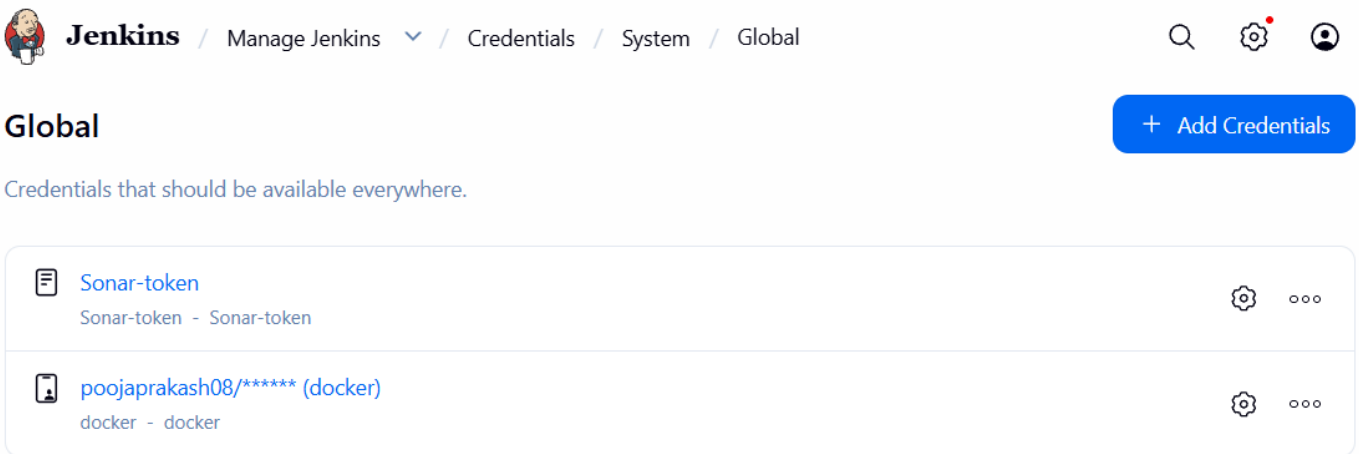

Jenkins / Manage Jenkins / Tools
Dependency-Check installations
+ Add Dependency-Check
Dependency-Check
Name
DP-Check
Install automatically
Install from github.com
Version
dependency-check 12.2.0


Jenkins / Manage Jenkins / Tools
Docker installations
+ Add Docker
Docker
Name
docker
Install automatically
Download from docker.com
Docker version
latest

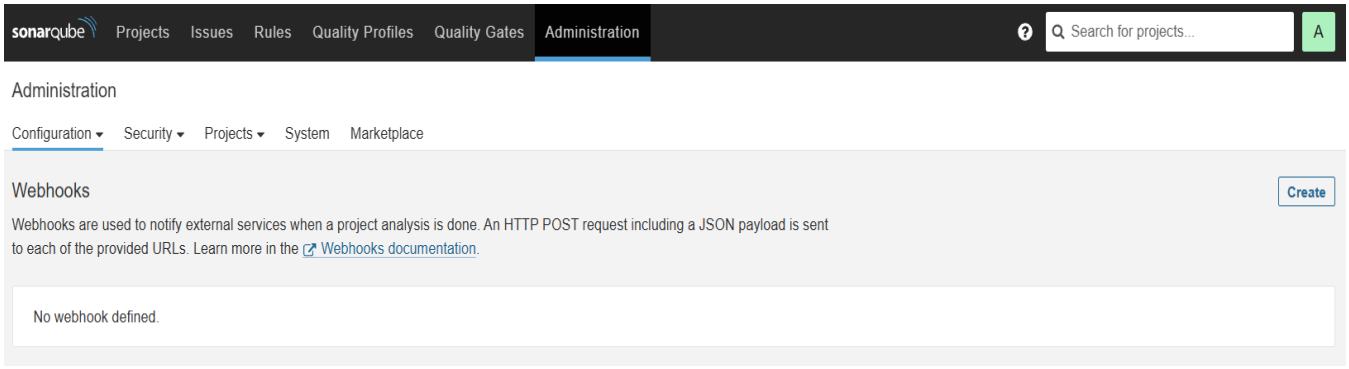
- Accessed SonarQube on port 9000, generated a token, and added it to Jenkins credentials.
- Added Docker Hub credentials in Jenkins.

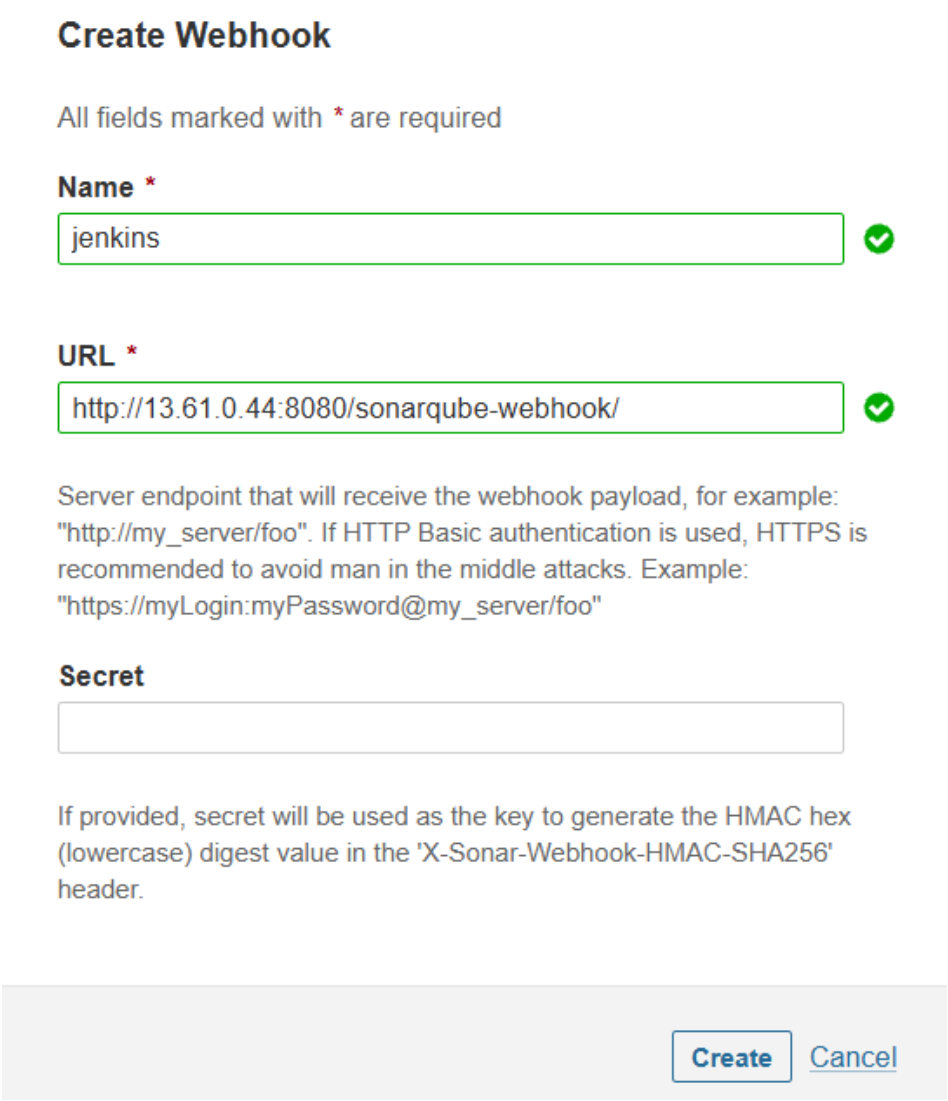




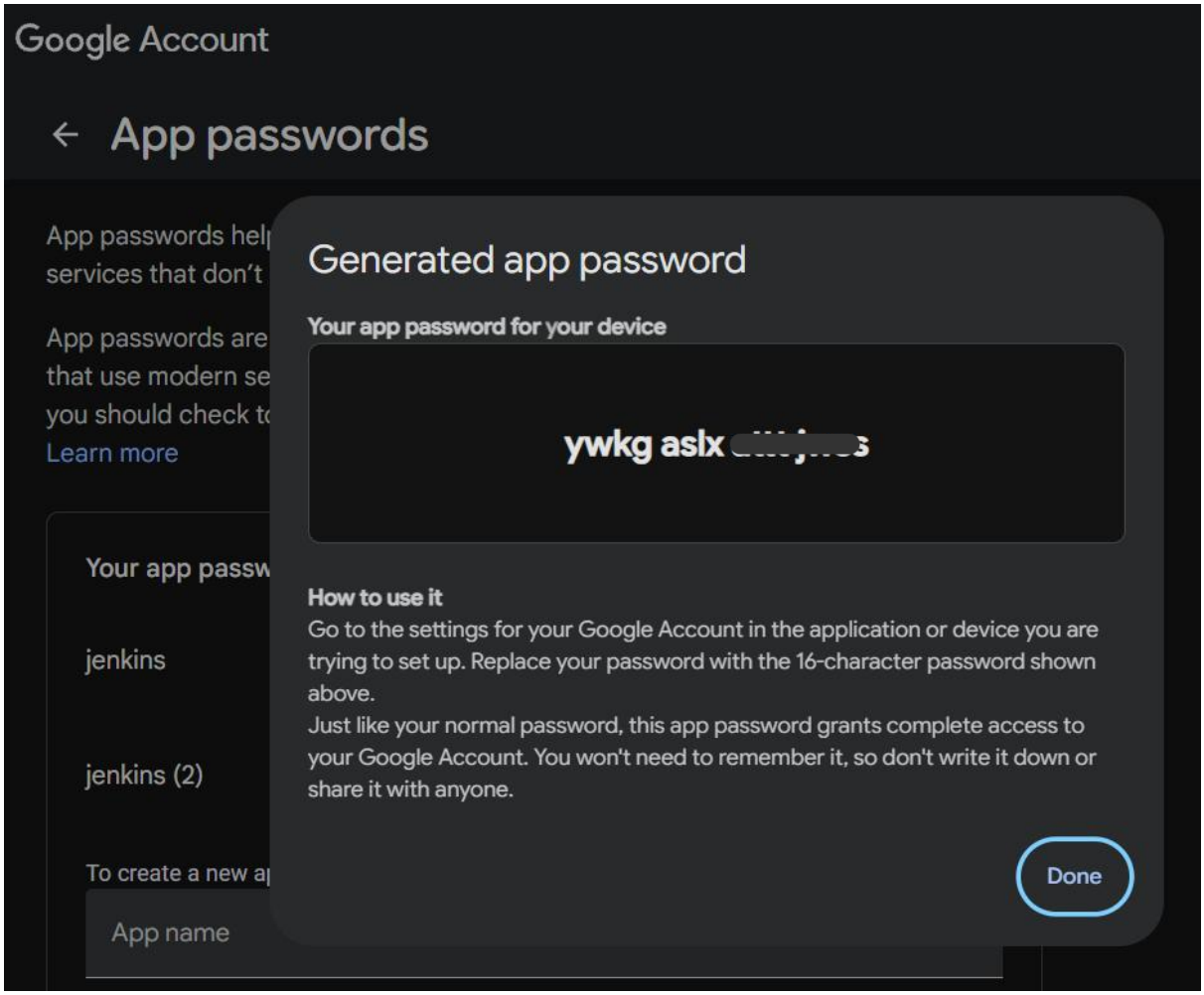


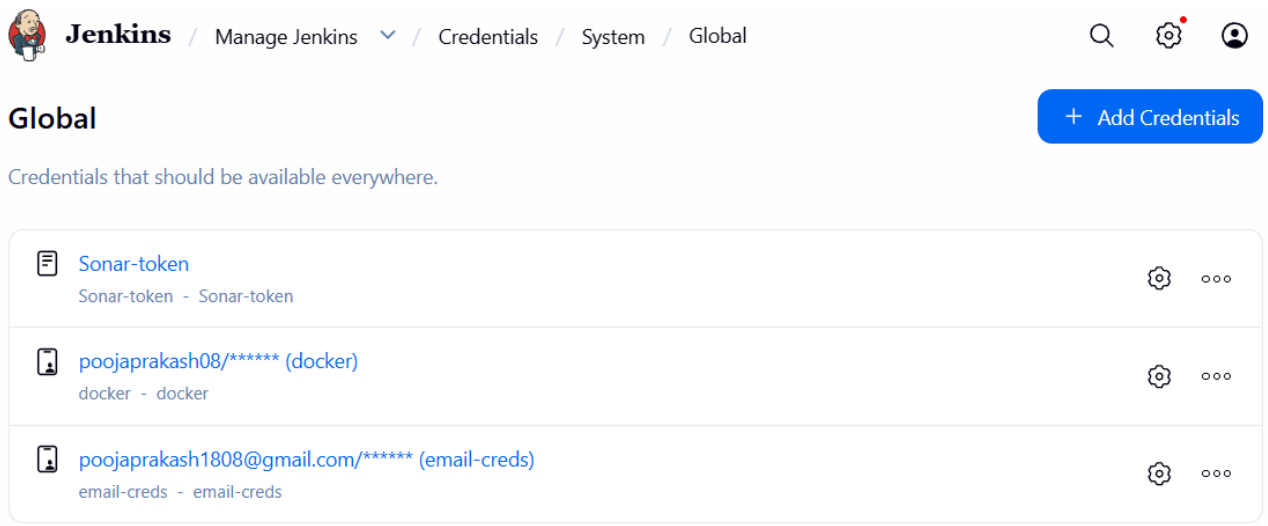
- Created a webhook in SonarQube to notify Jenkins when source code is updated.



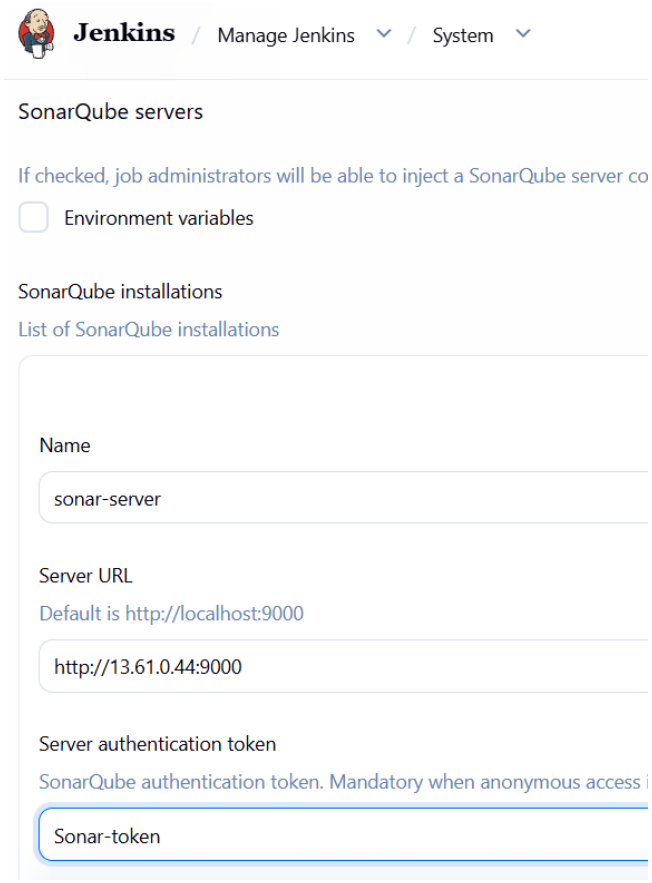


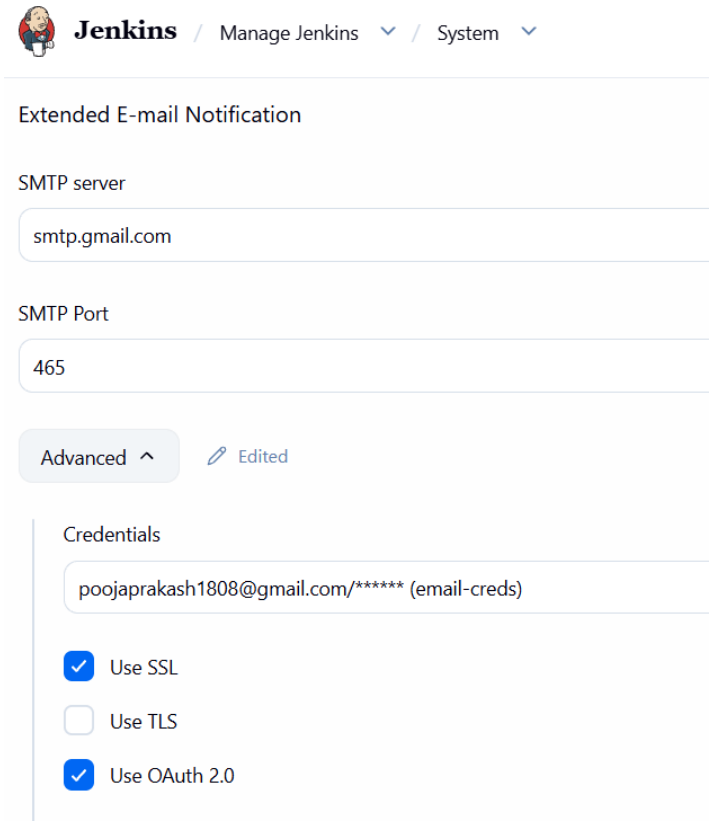
- Generated a Gmail App Password and added it to Jenkins credentials to enable email notifications for pipeline status.

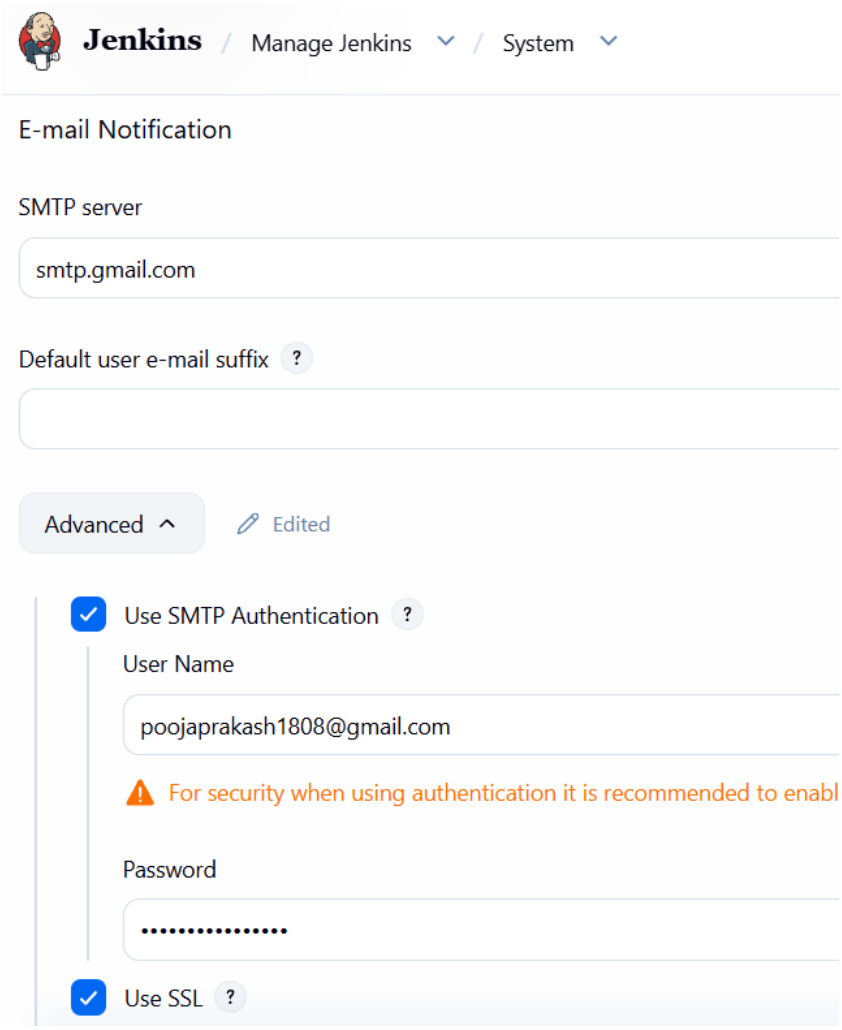


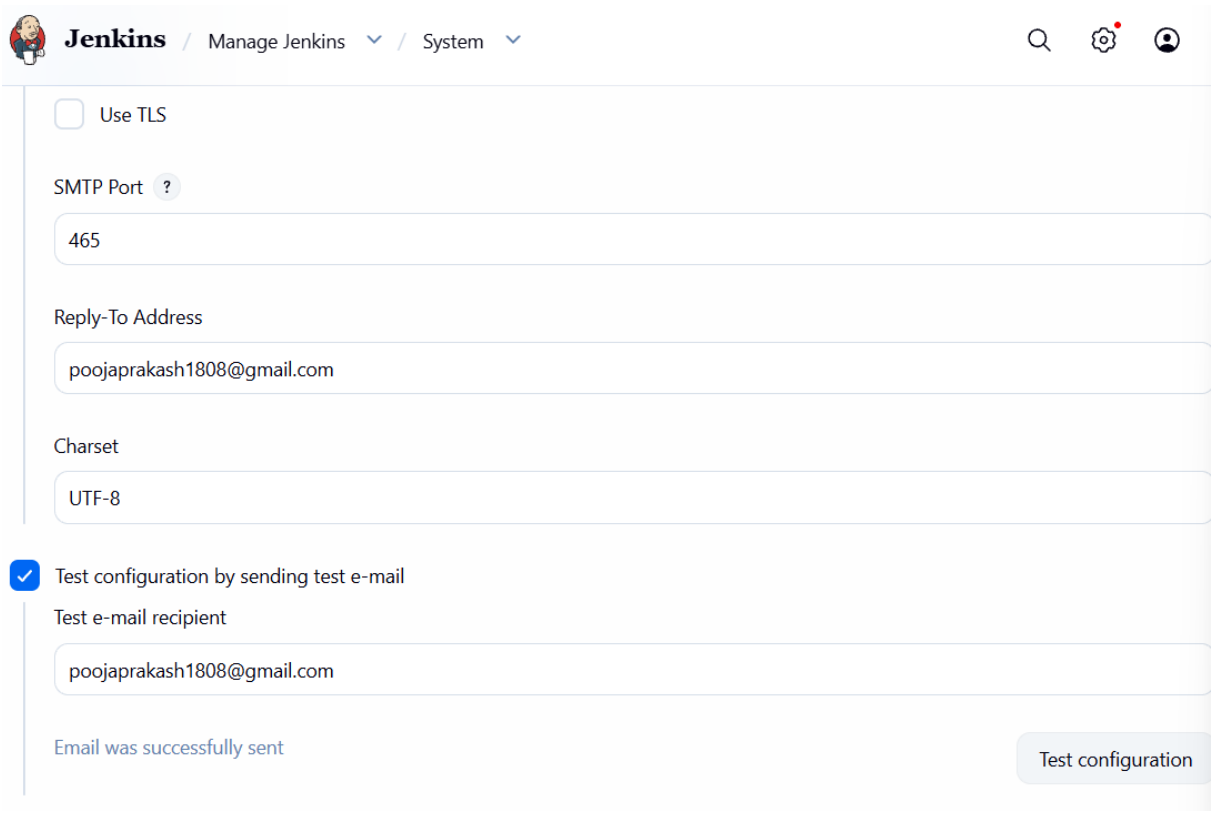


- Completed system configuration for SonarQube and Email Notifications in Jenkins.






Jenkins / Manage Jenkins / System
E-mail Notification
SMTP server
smtp.gmail.com
Default user e-mail suffix
Advanced
Edited
Use SMTP Authentication
User Name
poojaprakash1808@gmail.com
For security when using authentication it is recommended to enabl
Password
Use SSL


Jenkins / Manage Jenkins / System
Use TLS
SMTP Port
465
Reply-To Address
poojaprakash1808@gmail.com
Charset
UTF-8
Test configuration by sending test e-mail
Test e-mail recipient
poojaprakash1808@gmail.com
Email was successfully sent
Test configuration

3. CI/CD Pipeline Execution

- Created a Jenkinsfile to automate the pipeline stages: Clone source code from GitHub, Perform SonarQube code analysis, Install npm dependencies, Run OWASP dependency check, Perform Trivy file system scan, Build Docker image, Tag and push image to Docker Hub, Run Docker Scout image analysis, Deploy containerized application and Send email notifications

Welcome jenkinsfile

jenkinsfile

```
pipeline {
    agent any
    tools {
        jdk 'jdk17'
        nodejs 'node25'
    }
    environment {
        SCANNER_HOME=tool 'sonar-scanner'
    }
    stages {
        stage ("clean workspace") {
            steps {
                cleanWs()
            }
        }
        stage ("Git Checkout") {
            steps {
                git 'https://github.com/PoojaPrakash08/DevOps-Project-Zomato-Kastro.git'
            }
        }
        stage("Sonarqube Analysis"){
            steps{
                withSonarQubeEnv('sonar-server') {
                    sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=zomato \
                    -Dsonar.projectKey=zomato '''
                }
            }
        }
        stage("Code Quality Gate"){
           steps {
                script {
                    waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
                }
            }
        }
```

```
        stage("Install NPM Dependencies") {
            steps {
                sh "npm install"
            }
        }
        stage('OWASP FS SCAN') {
            steps {
                dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit -n', odcInstallation: 'DP-Check'
                dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
    }
}
        stage ("Trivy File Scan") {
            steps {
                sh "trivy fs . > trivy.txt"
            }
        }
        stage ("Build Docker Image") {
            steps {
                sh "docker build -t zomato ."
            }
        }
        stage ("Tag & Push to DockerHub") {
            steps {
                script {
                    withDockerRegistry(credentialsId: 'docker') {
                        sh "docker tag zomato poojaprakash08/zomato:latest "
                        sh "docker push poojaprakash08/zomato:latest "
                    }
                }
            }
        }
```

```
            stage('Docker Scout Image') {
                steps {
                    script{
                       withDockerRegistry(credentialsId: 'docker', toolName: 'docker'){
                            sh 'docker-scout quickview poojaprakash08/zomato:latest'
                            sh 'docker-scout cves poojaprakash08/zomato:latest'
                            sh 'docker-scout recommendations poojaprakash08/zomato:latest'
                       }
                    }
                }
            }
            stage ("Deploy to Container") {
                steps {
                    sh 'docker run -d --name zomato -p 3000:3000 poojaprakash08/zomato:latest'
                }
            }
        }
```

```
    post {
    always {
        emailext attachLog: true,
            subject: "'${currentBuild.result}'",
            body: """
                <html>
                <body>
                    <div style="background-color: #FFA07A; padding: 10px; margin-bottom: 10px;">
                        <p style="color: white; font-weight: bold;">Project: ${env.JOB_NAME}</p>
                    </div>
                    <div style="background-color: #90EE90; padding: 10px; margin-bottom: 10px;">
                        <p style="color: white; font-weight: bold;">Build Number: ${env.BUILD_NUMBER}</p>
                    </div>
                    <div style="background-color: #87CEEB; padding: 10px; margin-bottom: 10px;">
                        <p style="color: white; font-weight: bold;">URL: ${env.BUILD_URL}</p>
                    </div>
                </body>
                </html>
            """,
            to: 'poojaprakash1808@gmail.com',
            mimeType: 'text/html',
            attachmentsPattern: 'trivy.txt'
    }
    }
}

```

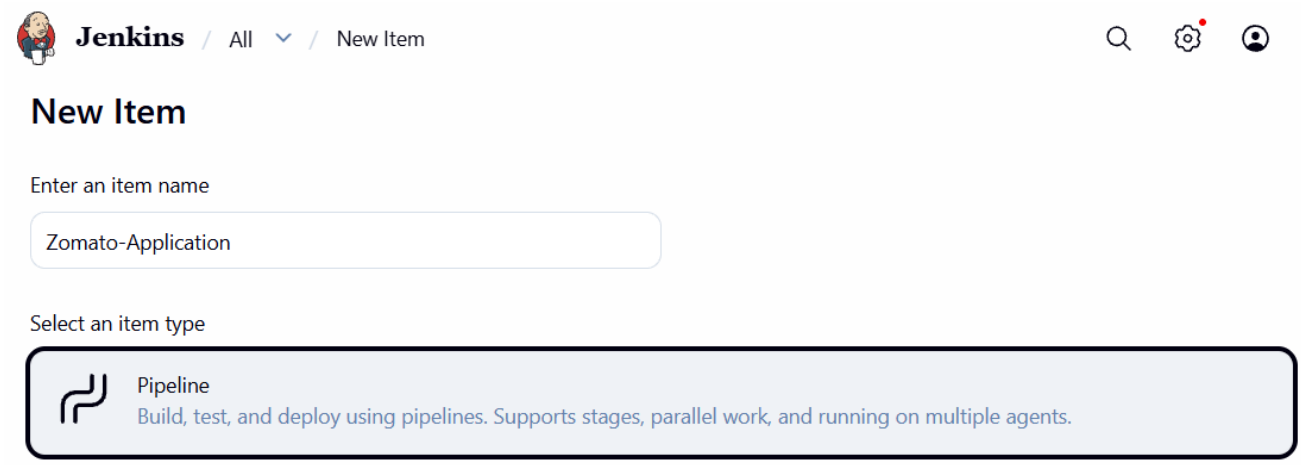
- Created a Dockerfile to build the application image.

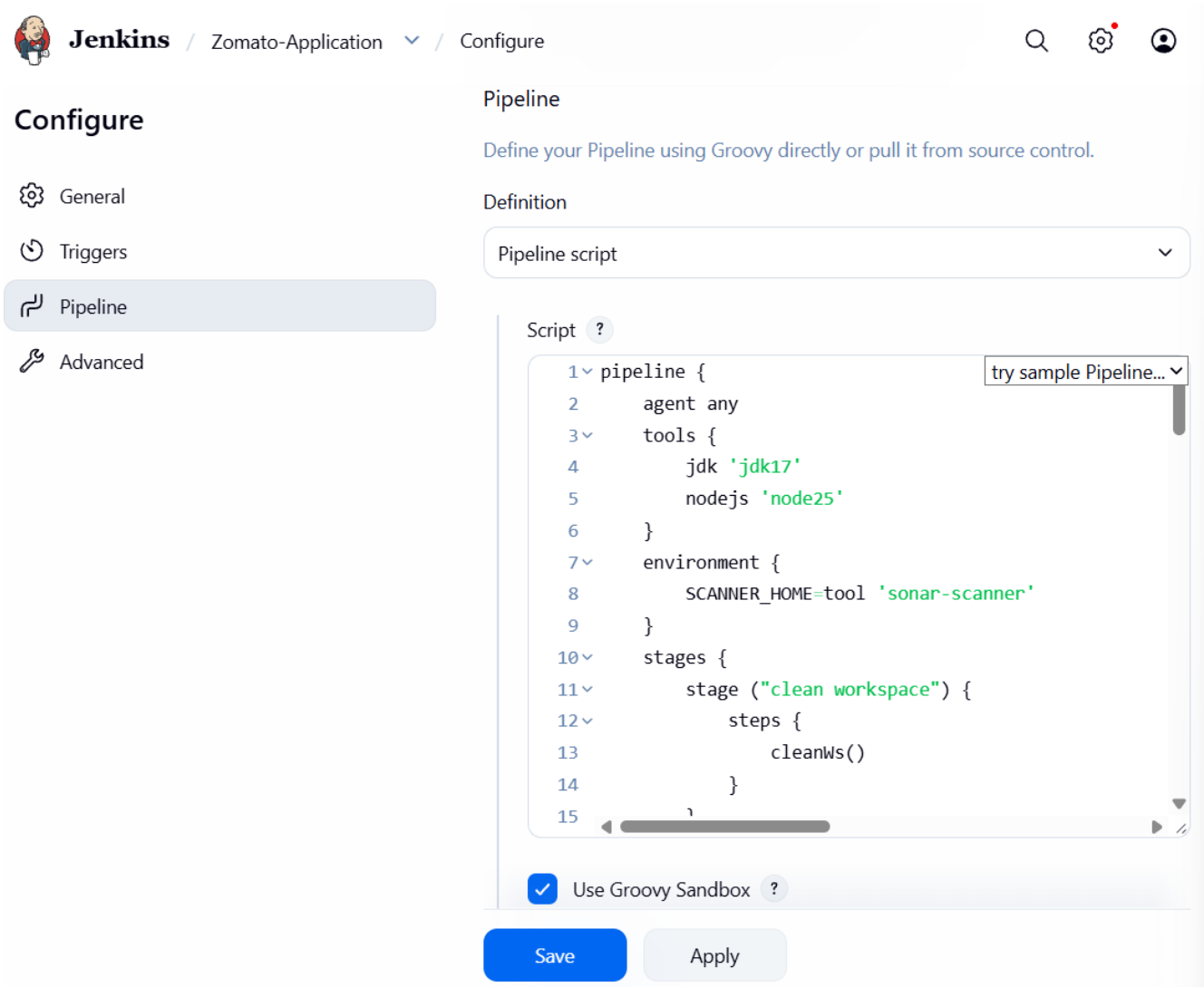
Welcome | jenkinsfile | Dockerfile

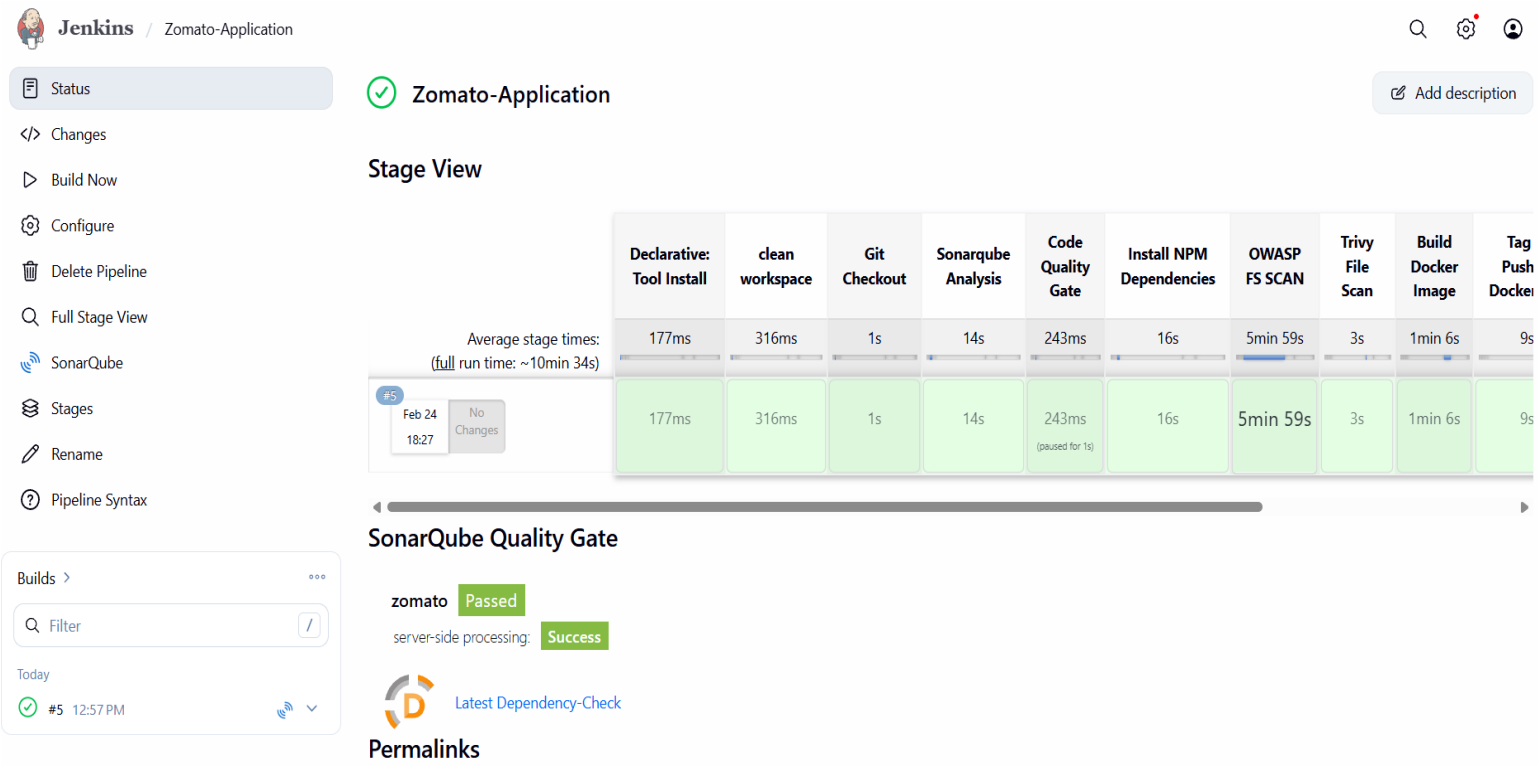
Dockerfile

```
# Use Node.js 16 slim as the base image
FROM node:16-slim

# Set the working directory
WORKDIR /app

# Copy package.json and package-lock.json to the working directory
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Build the React app
RUN npm run build

# Expose port 3000 (or the port your app is configured to listen on)
EXPOSE 3000

# Start your Node.js server (assuming it serves the React app)
CMD ["npm", "start"]

```

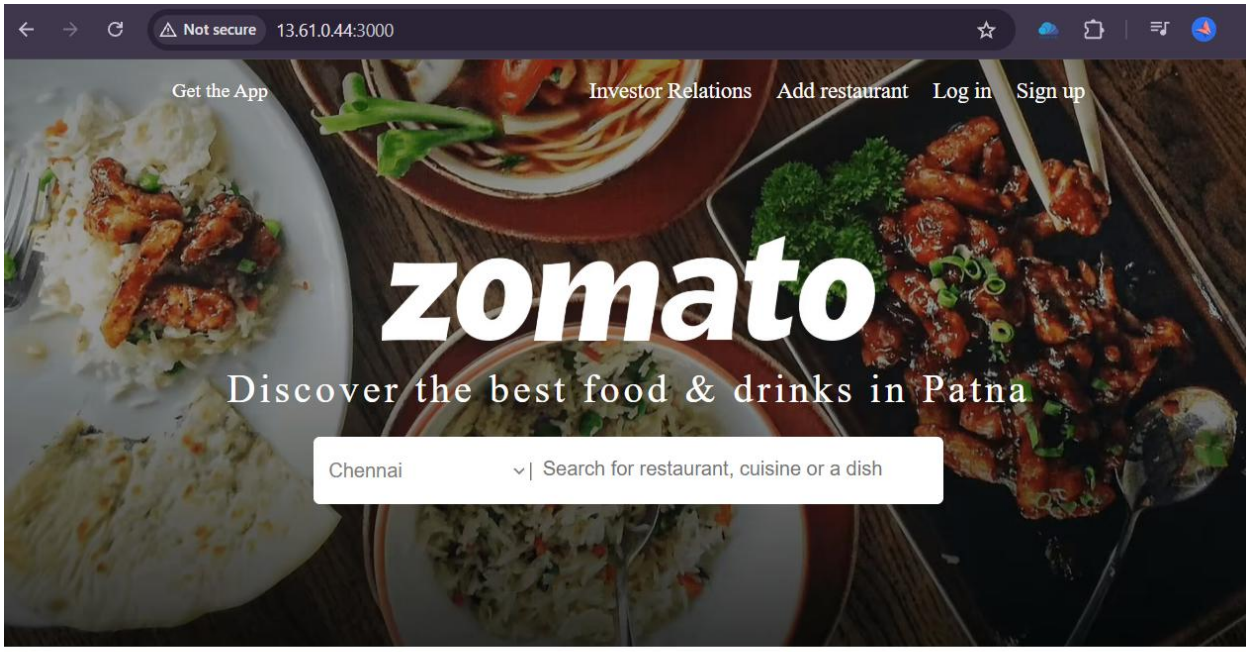
- Created a Jenkins Pipeline job named Zomato Application and executed the build.







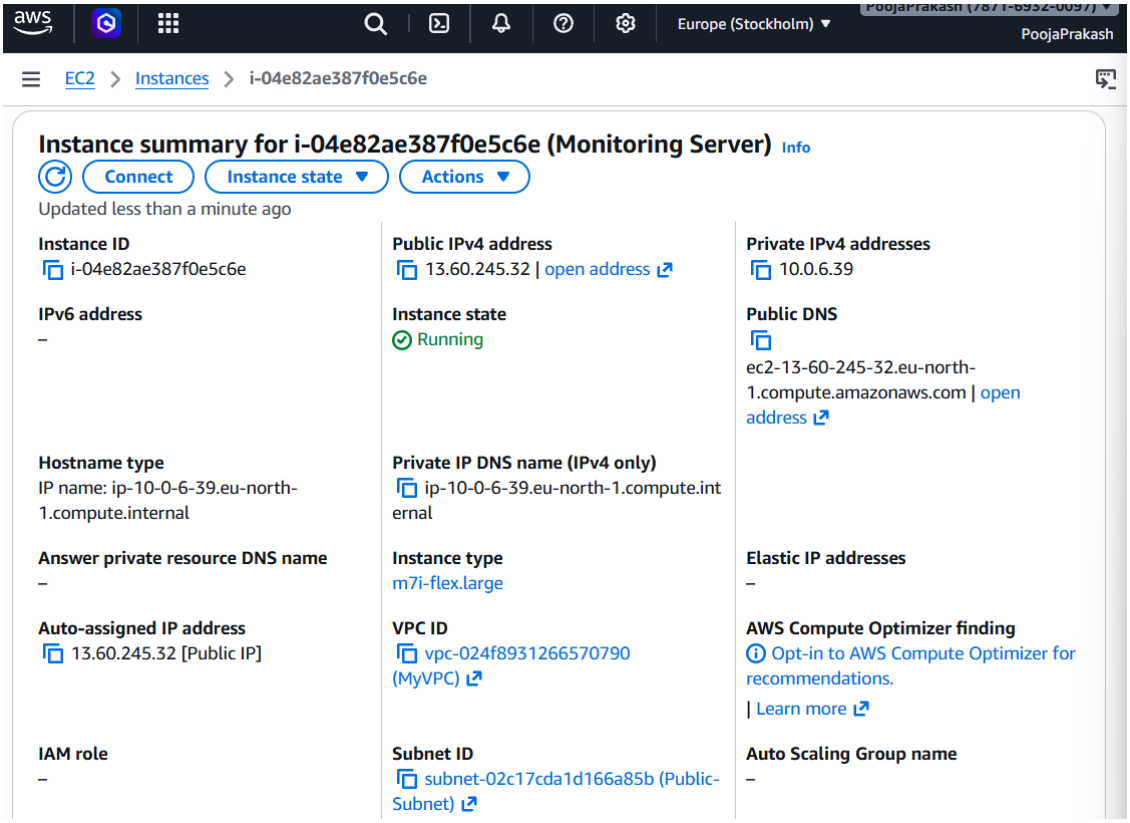
- Successfully deployed the Docker containerized Zomato Clone application.



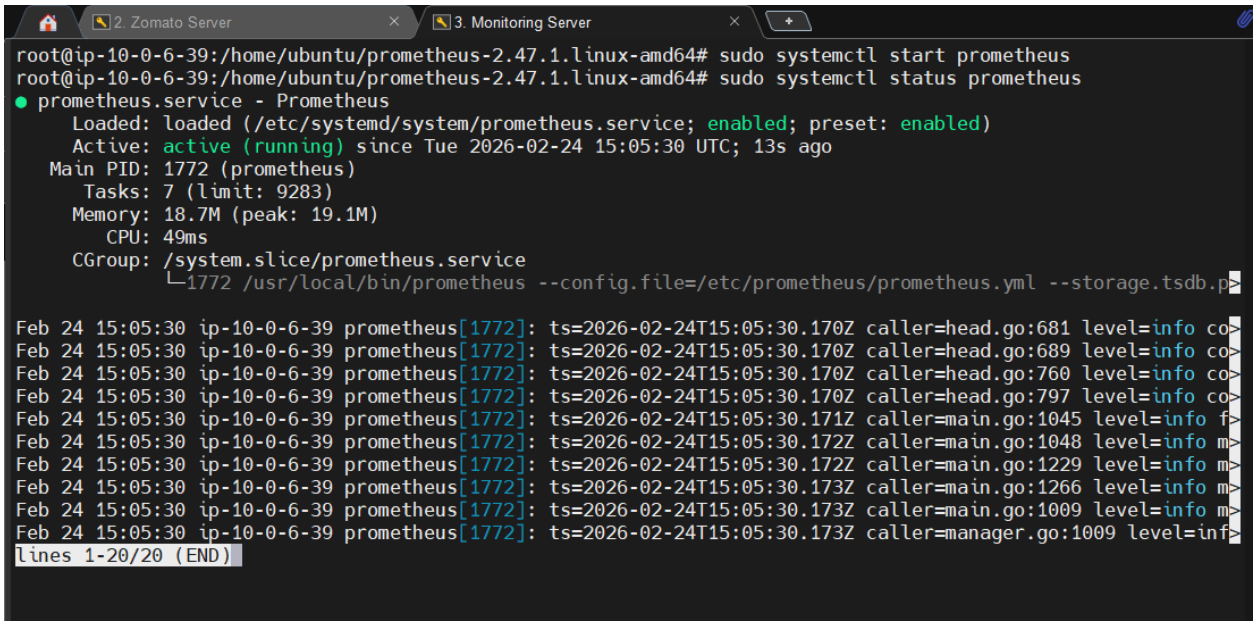
Phase 2: Monitoring with Prometheus and Grafana

1. Monitoring Server Setup

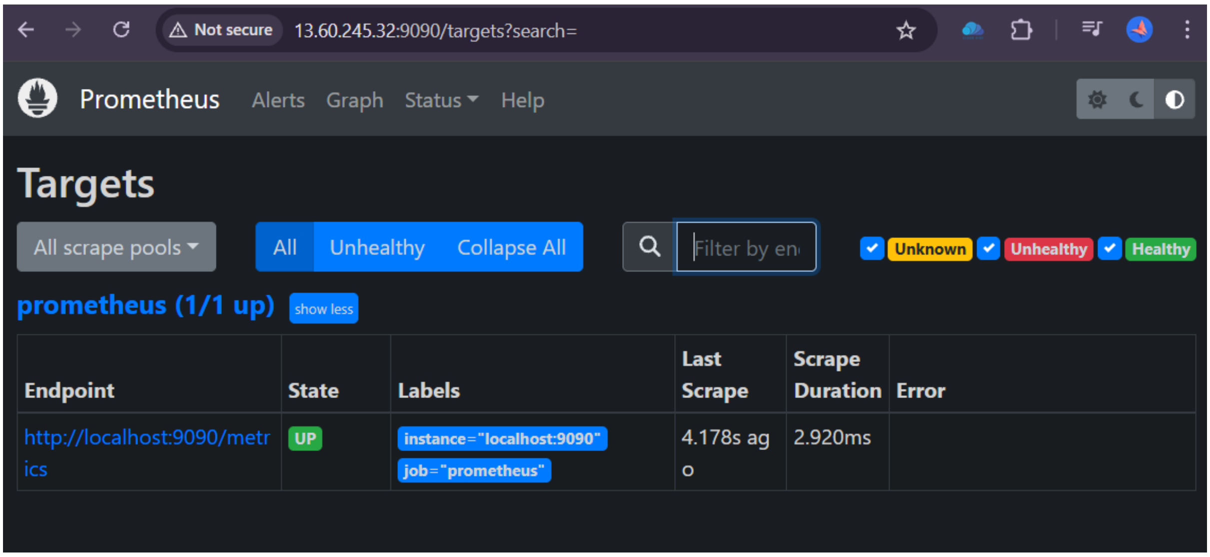
- Launched another Ubuntu EC2 instance named **Monitoring Server**.



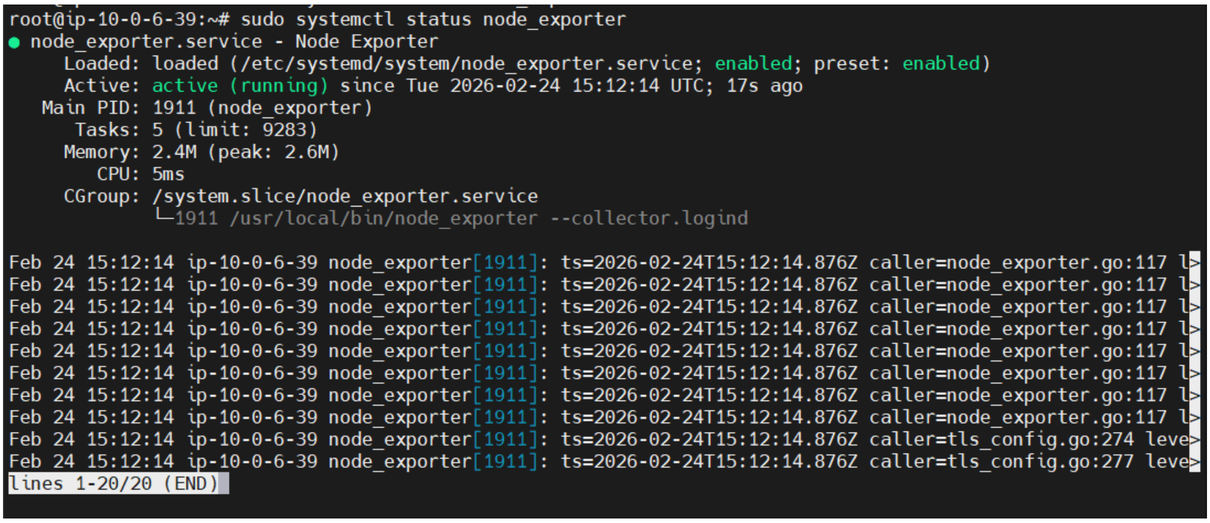
- Installed Prometheus on the Monitoring Server.



- Accessed Prometheus on port 9090.



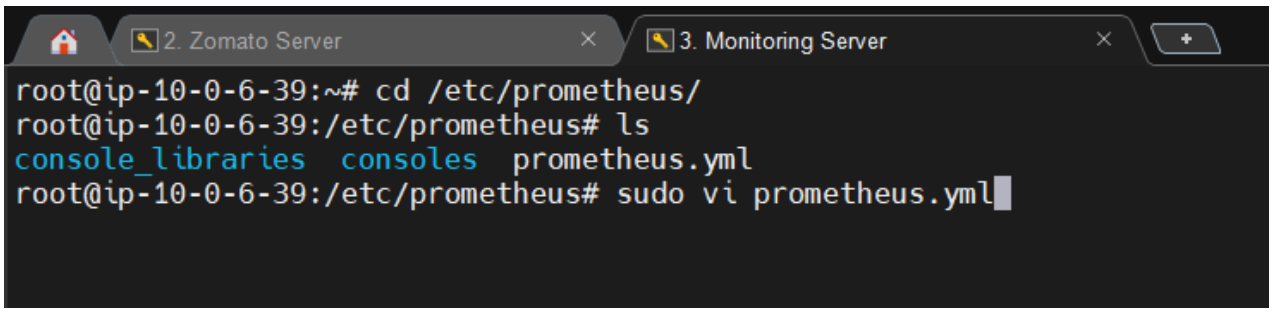
- Installed Node Exporter to collect system metrics.

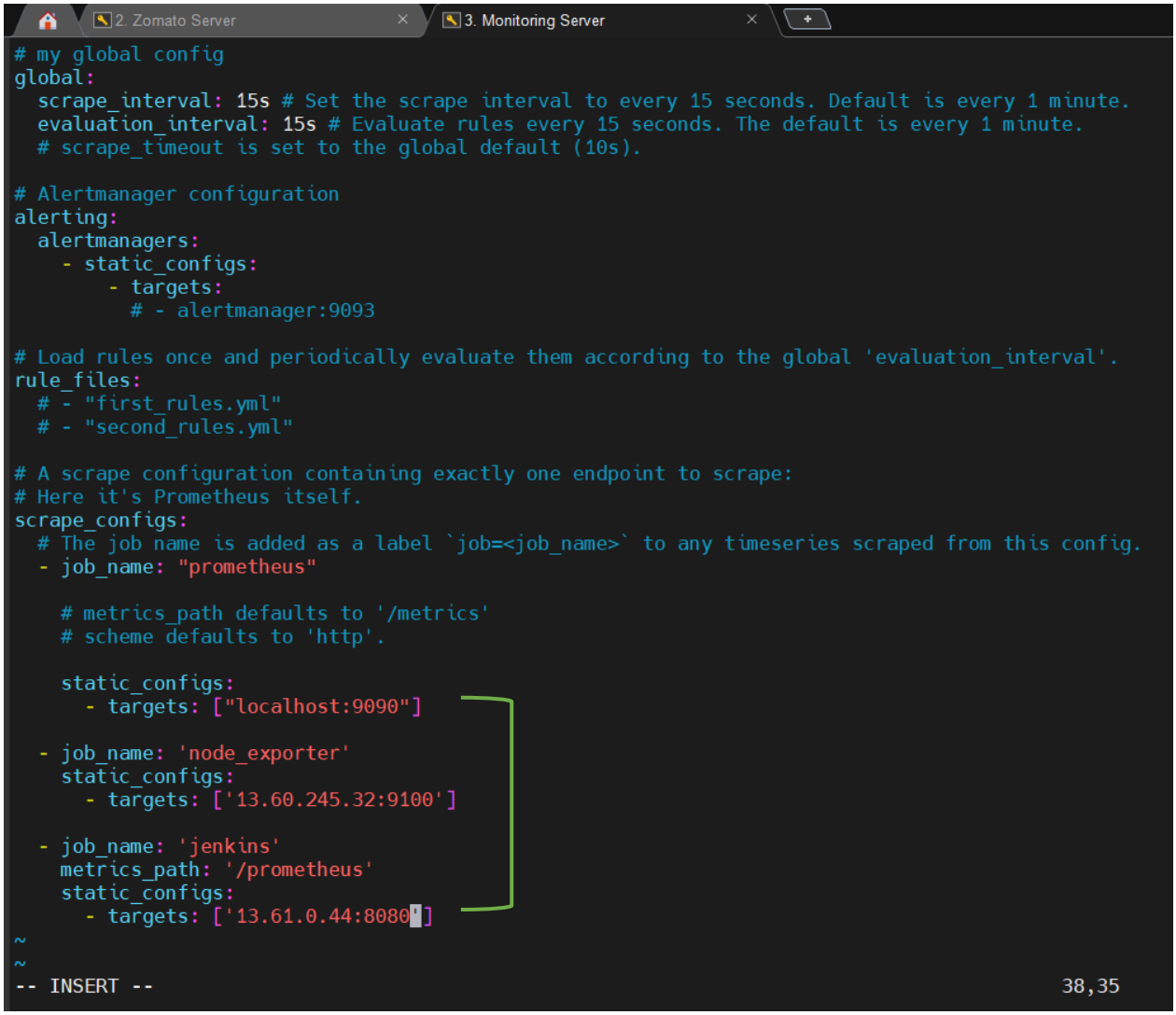


```
root@ip-10-0-6-39:~# sudo systemctl status node_exporter
● node_exporter.service - Node Exporter
     Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-02-24 15:12:14 UTC; 17s ago
   Main PID: 1911 (node_exporter)
      Tasks: 5 (limit: 9283)
     Memory: 2.4M (peak: 2.6M)
        CPU: 5ms
     CGroup: /system.slice/node_exporter.service
             └─1911 /usr/local/bin/node_exporter --collector.logind

Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=tls_config.go:274 leve>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=tls_config.go:277 leve>
lines 1-20/20 (END)
```

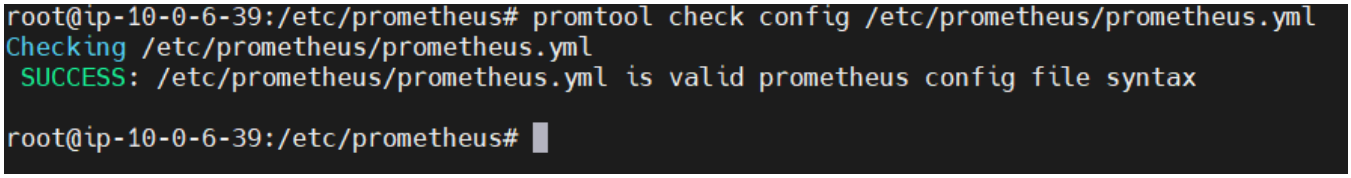
2. Prometheus Configuration

- Modified the prometheus.yml file to scrape metrics from:
  - Node Exporter
  - Jenkins

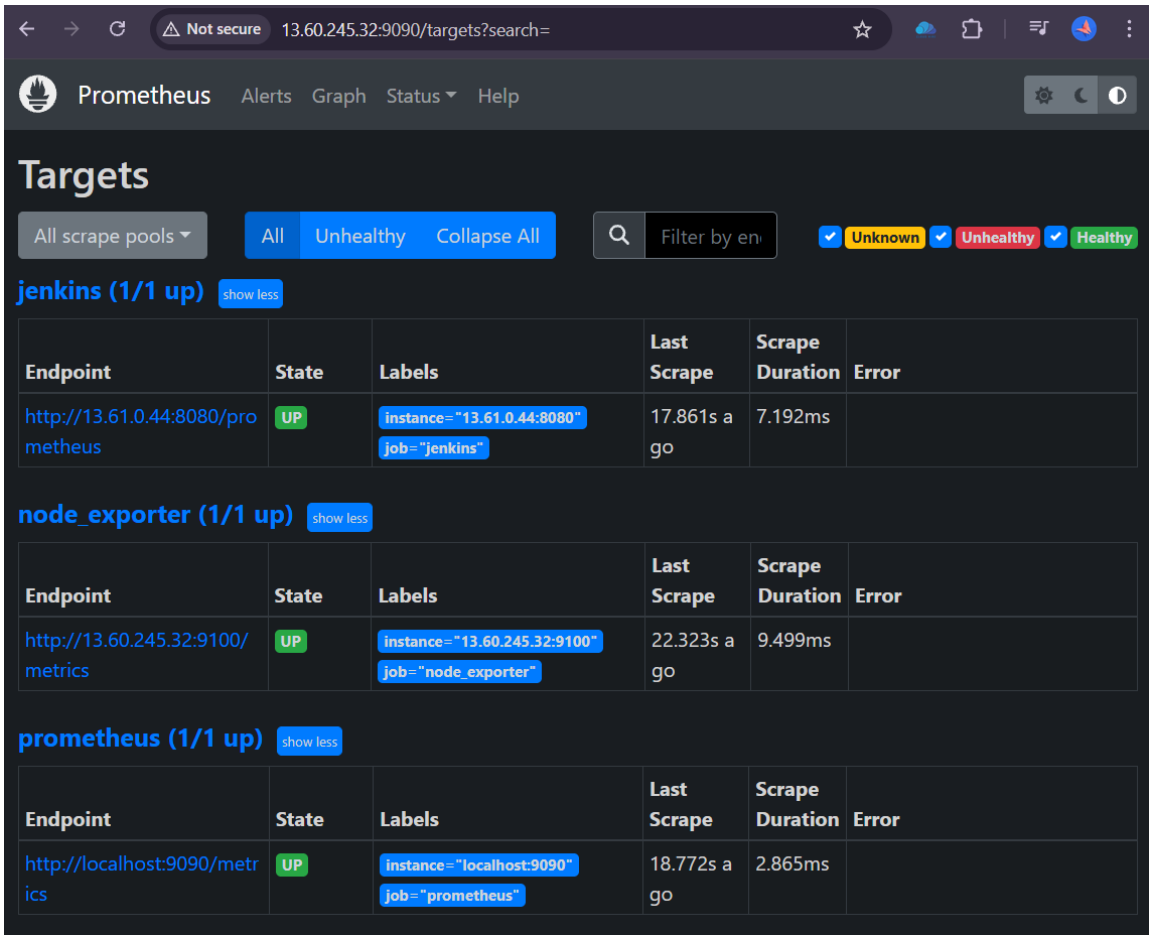




- Validated the configuration using promtool.

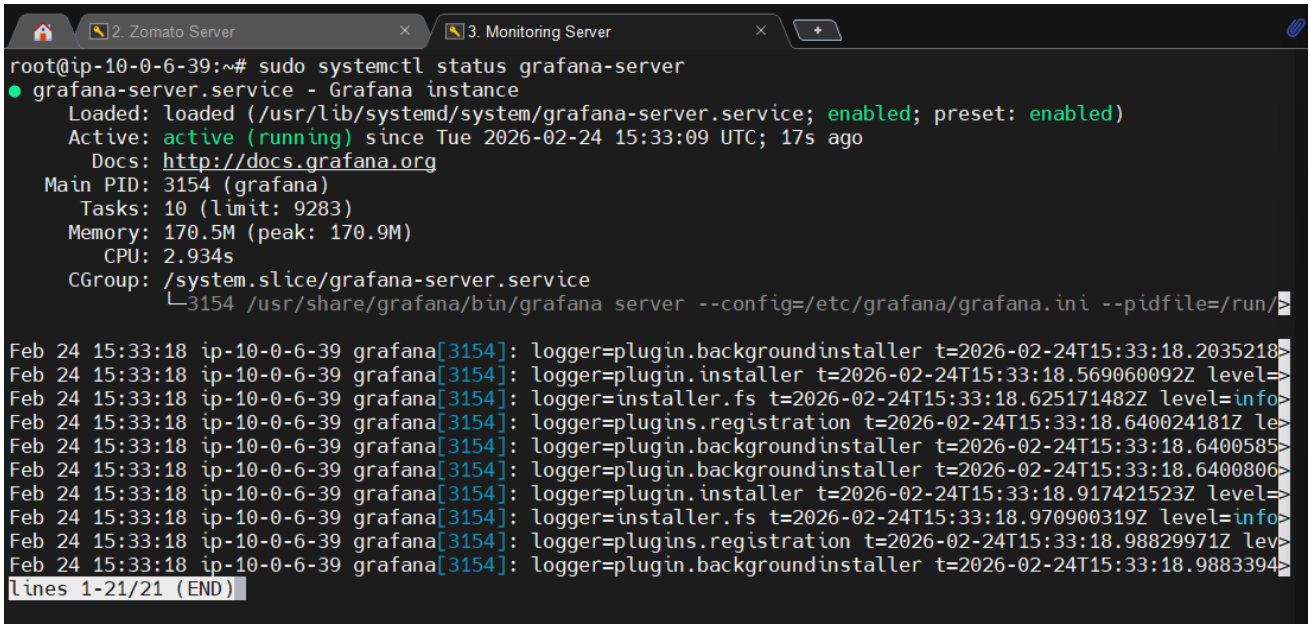


- Reloaded Prometheus and verified targets were up.

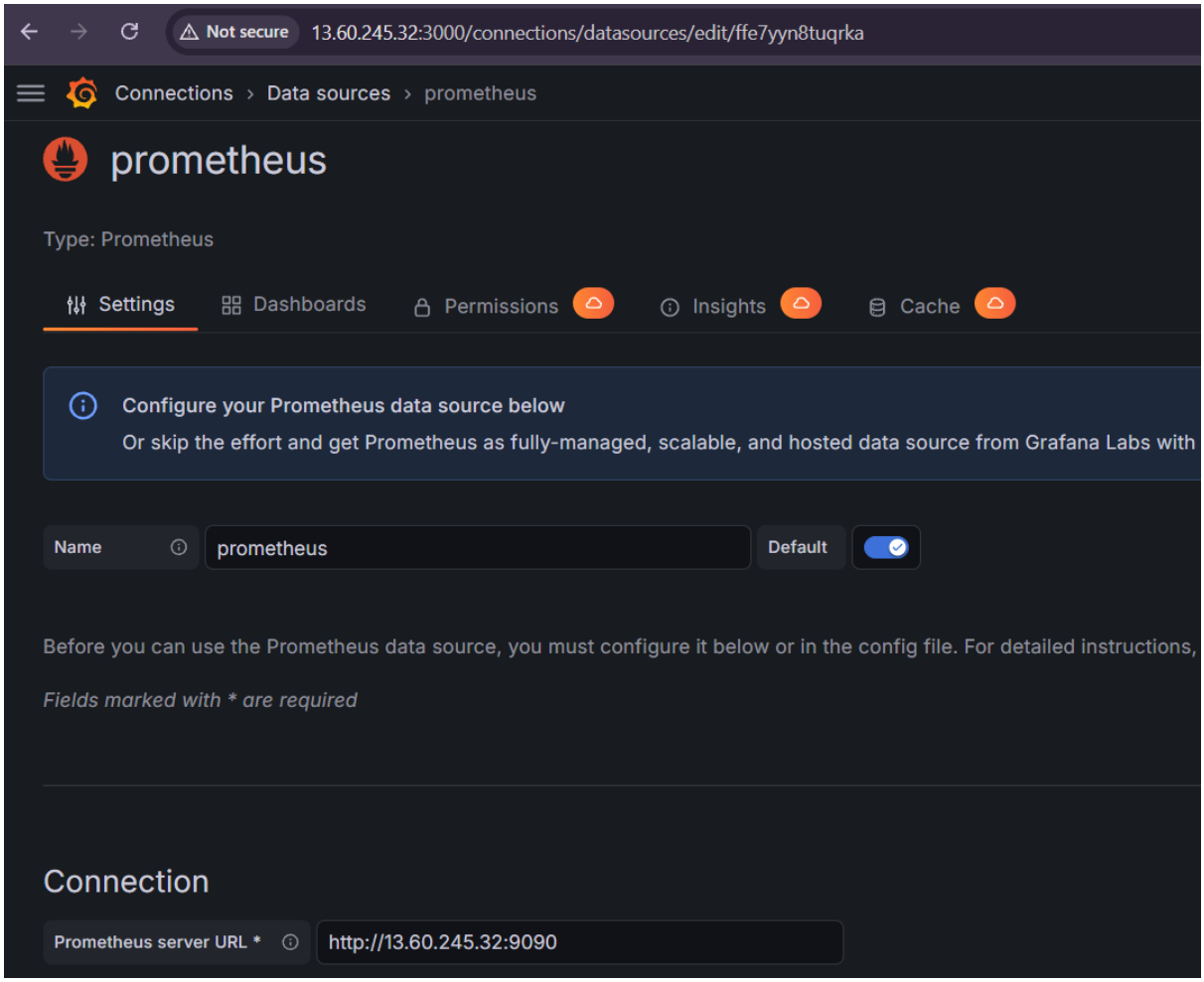


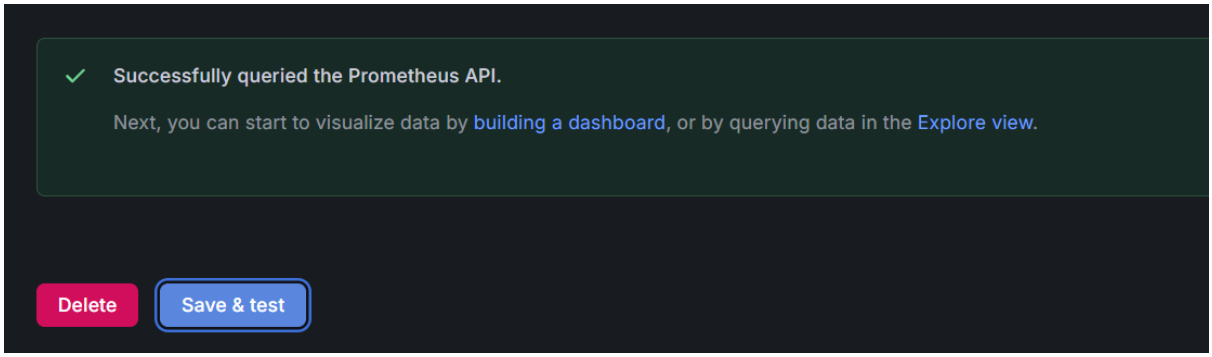
3. Grafana Visualization

- Installed Grafana on the Monitoring Server.

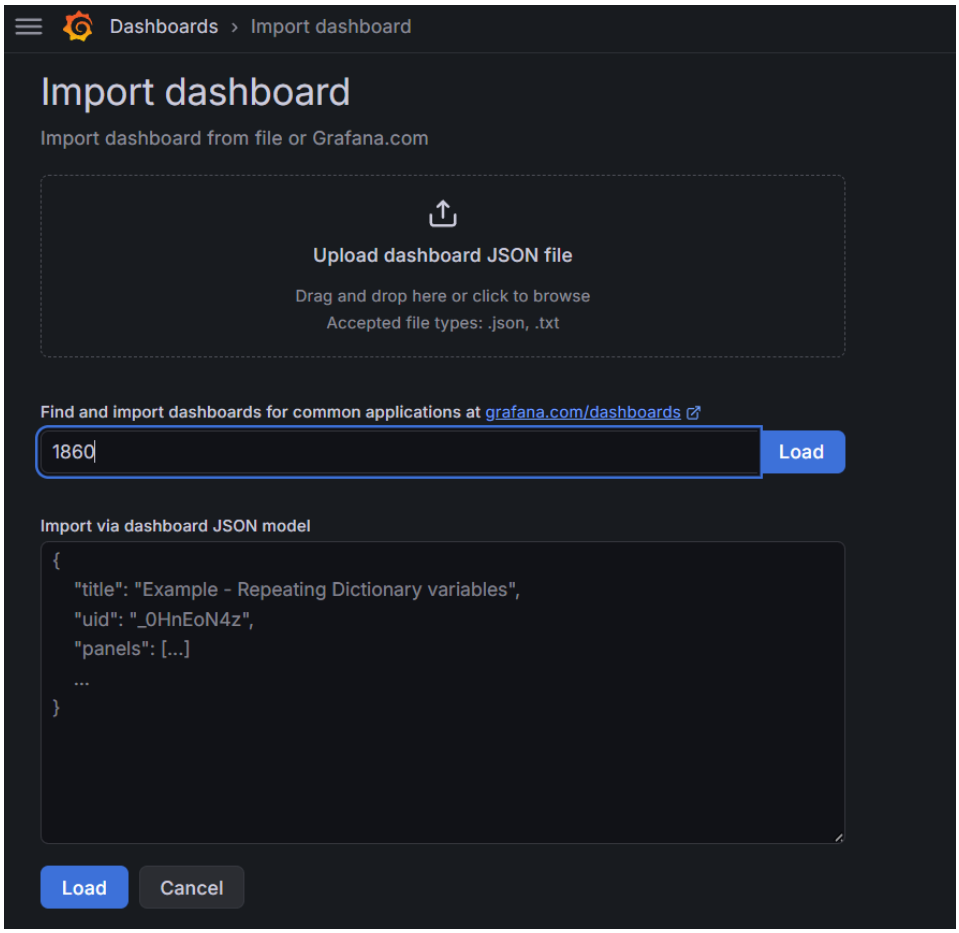


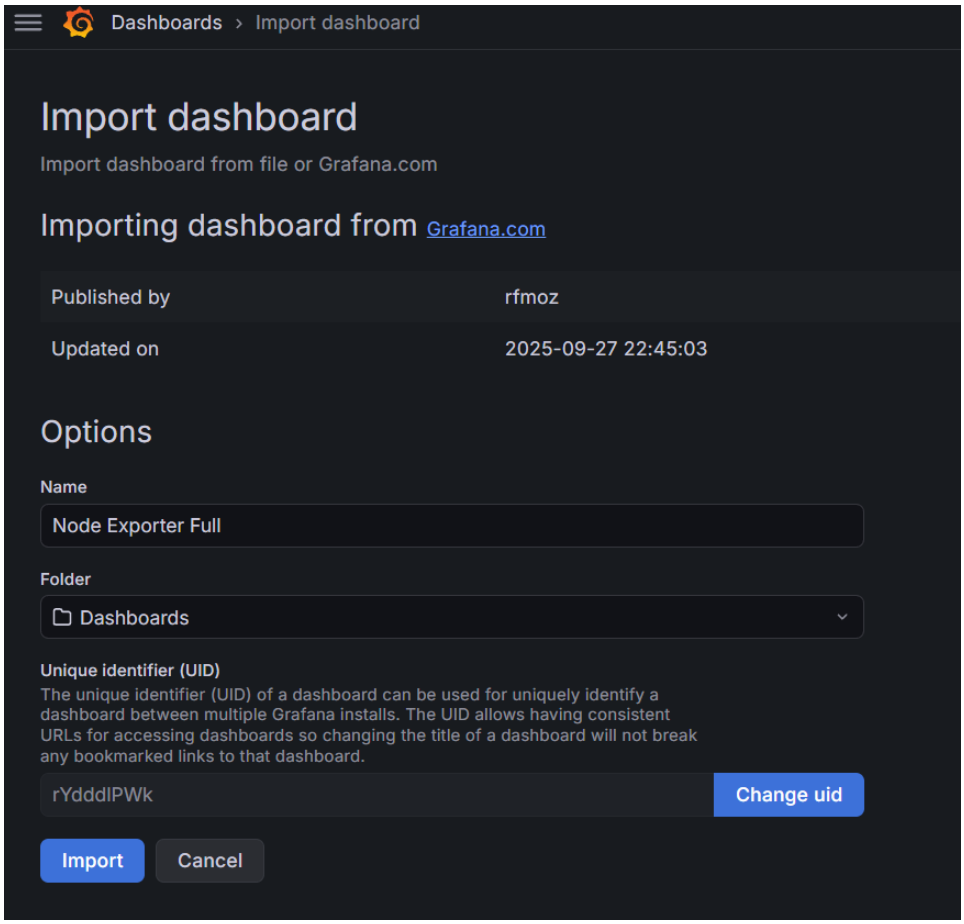
- Accessed Grafana on port 3000 and added Prometheus as a Data Source.

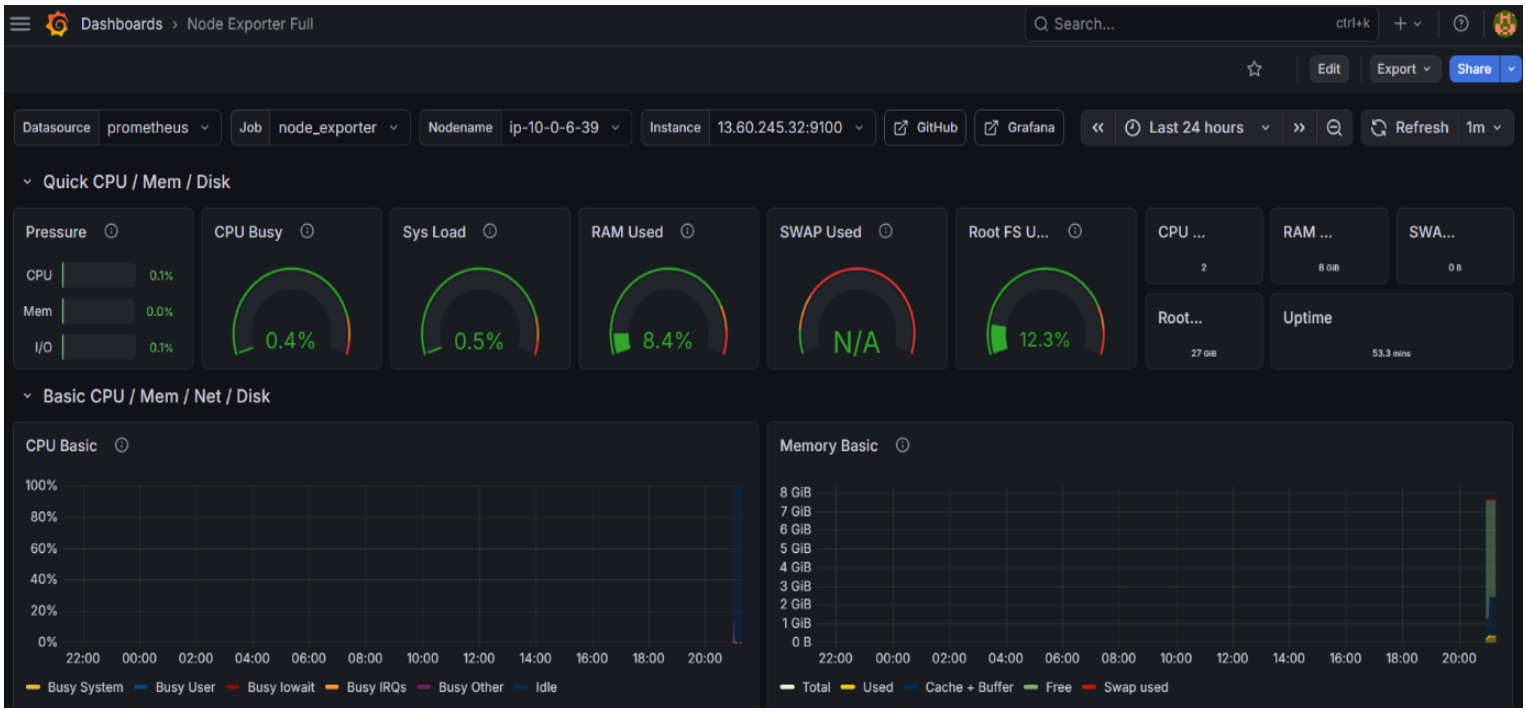


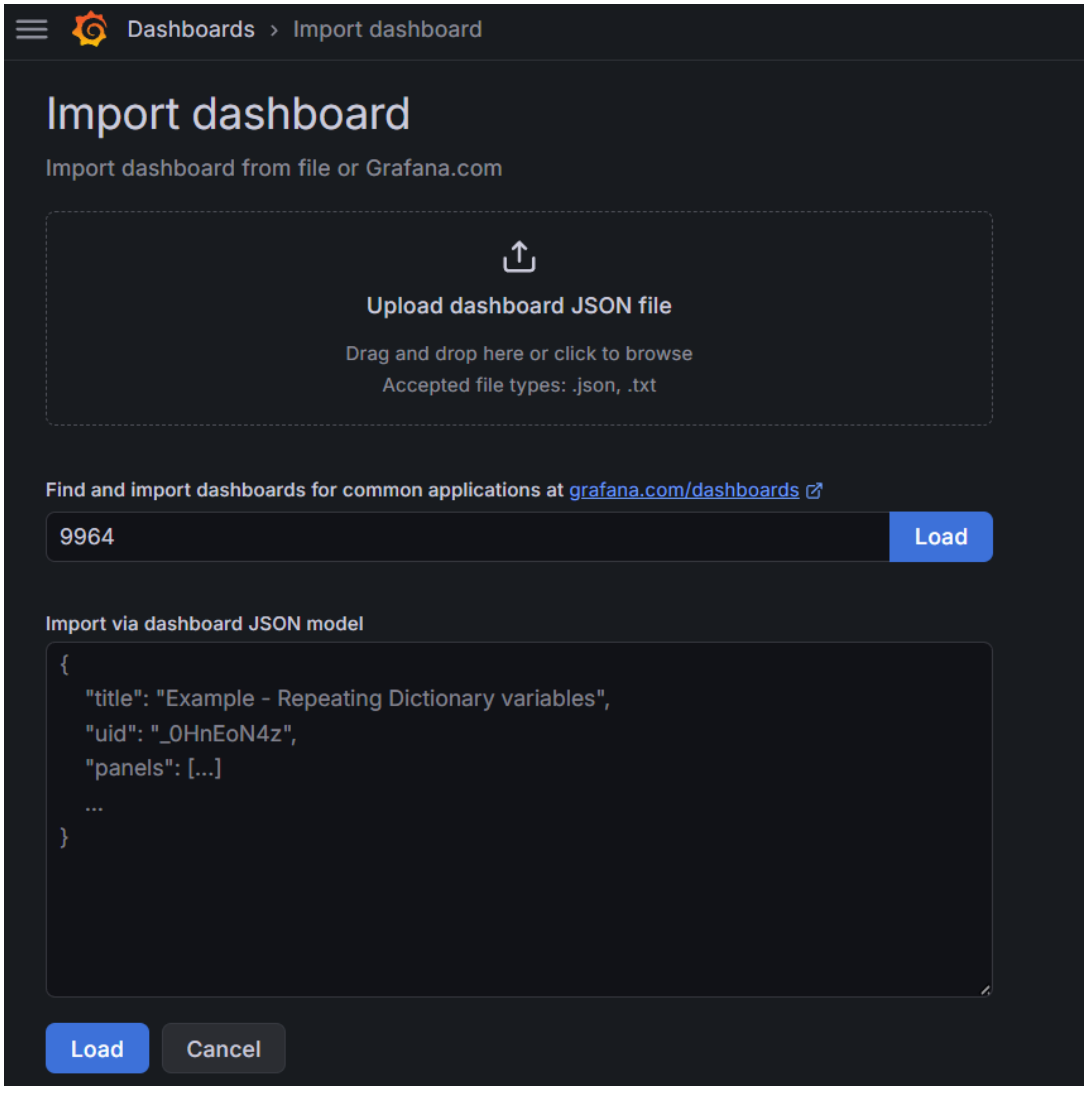


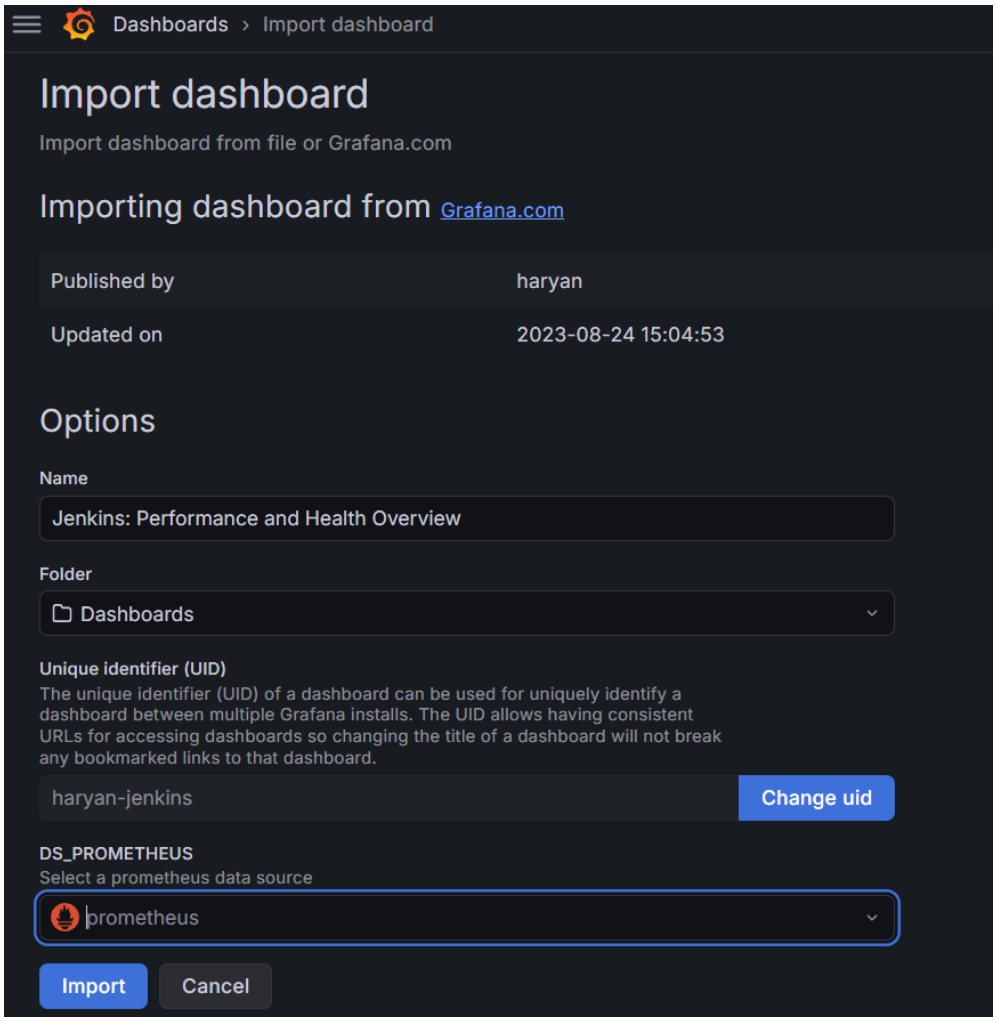
- Imported official dashboards for:
  - Node Exporter
  - Jenkins
- These dashboards provided visual representation of system and pipeline metrics.

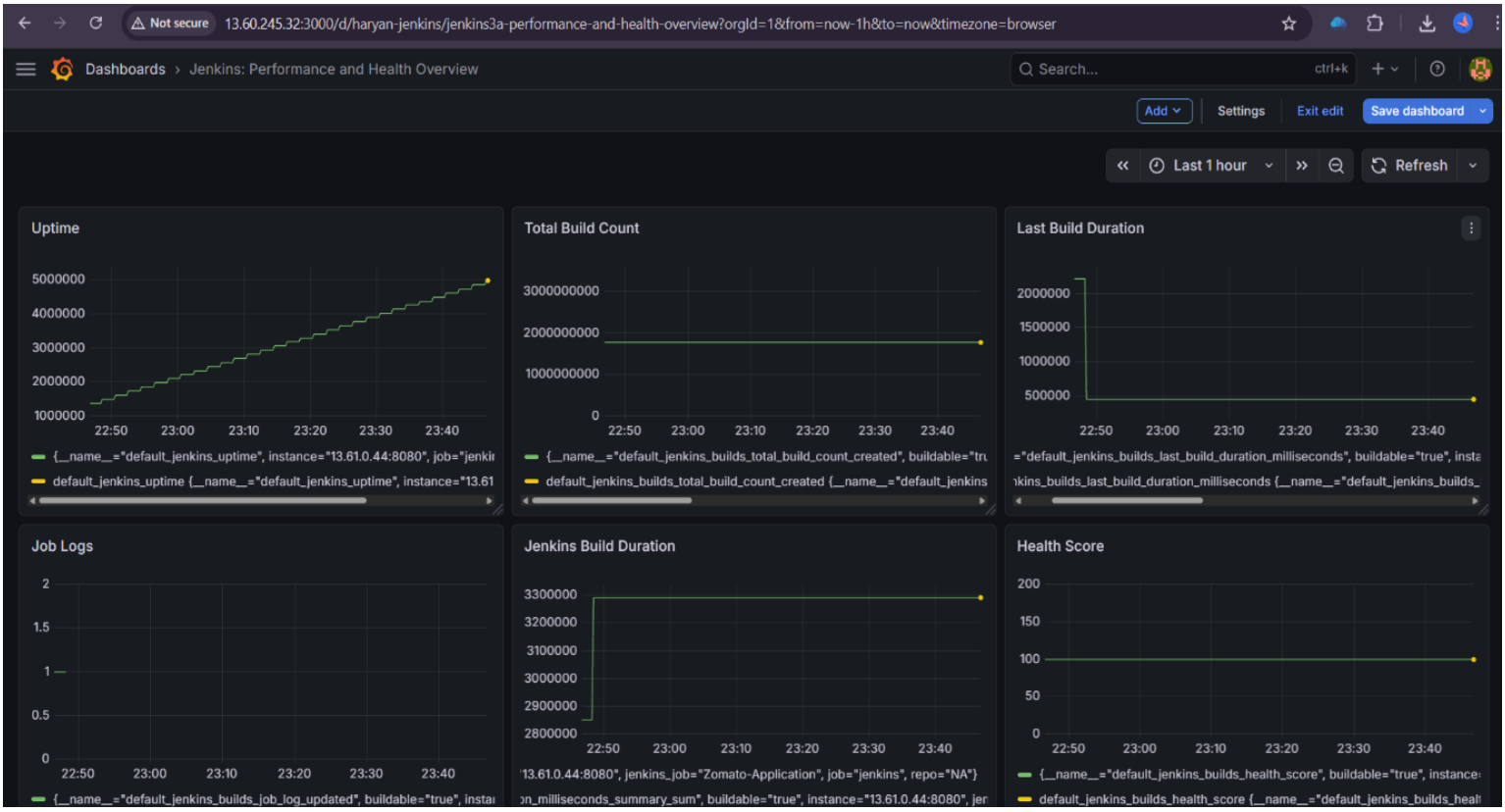

Dashboards › Import dashboard
Import dashboard
Import dashboard from file or Grafana.com
Upload dashboard JSON file
Drag and drop here or click to browse
Accepted file types: .json, .txt
Find and import dashboards for common applications at grafana.com/dashboards
1860
Load
Import via dashboard JSON model
{
"title": "Example - Repeating Dictionary variables",
"uid": "_0HnEoN4z",
"panels": [...]
...
}
Load
Cancel


Dashboards › Import dashboard
Import dashboard
Import dashboard from file or Grafana.com
Importing dashboard from Grafana.com
Published by
rfmoz
Updated on
2025-09-27 22:45:03
Options
Name
Node Exporter Full
Folder
Dashboards
Unique identifier (UID)
The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
rYdddlPWk
Change uid
Import
Cancel


Dashboards › Node Exporter Full
Search...
ctrl+k
Edit
Export
Share
Datasource prometheus
Job node_exporter
Nodename ip-10-0-6-39
Instance 13.60.245.32:9100
GitHub
Grafana
Last 24 hours
Refresh 1m
Quick CPU / Mem / Disk
Pressure
CPU 0.1%
Mem 0.0%
I/O 0.1%
CPU Busy
0.4%
Sys Load
0.5%
RAM Used
8.4%
SWAP Used
N/A
Root FS U...
12.3%
CPU ...
RAM ...
SWA...
Root...
Uptime
Basic CPU / Mem / Net / Disk
CPU Basic
100%
80%
60%
40%
20%
0%
22:00 00:00 02:00 04:00 06:00 08:00 10:00 12:00 14:00 16:00 18:00 20:00
Busy System
Busy User
Busy Iowait
Busy IRQs
Busy Other
Idle
Memory Basic
8 GiB
7 GiB
6 GiB
5 GiB
4 GiB
3 GiB
2 GiB
1 GiB
0 B
22:00 00:00 02:00 04:00 06:00 08:00 10:00 12:00 14:00 16:00 18:00 20:00
Total
Used
Cache + Buffer
Free
Swap used


Dashboards › Import dashboard
Import dashboard
Import dashboard from file or Grafana.com
Upload dashboard JSON file
Drag and drop here or click to browse
Accepted file types: .json, .txt
Find and import dashboards for common applications at grafana.com/dashboards
9964
Load
Import via dashboard JSON model
{
"title": "Example - Repeating Dictionary variables",
"uid": "_0HnEoN4z",
"panels": [...]
...
}
Load
Cancel

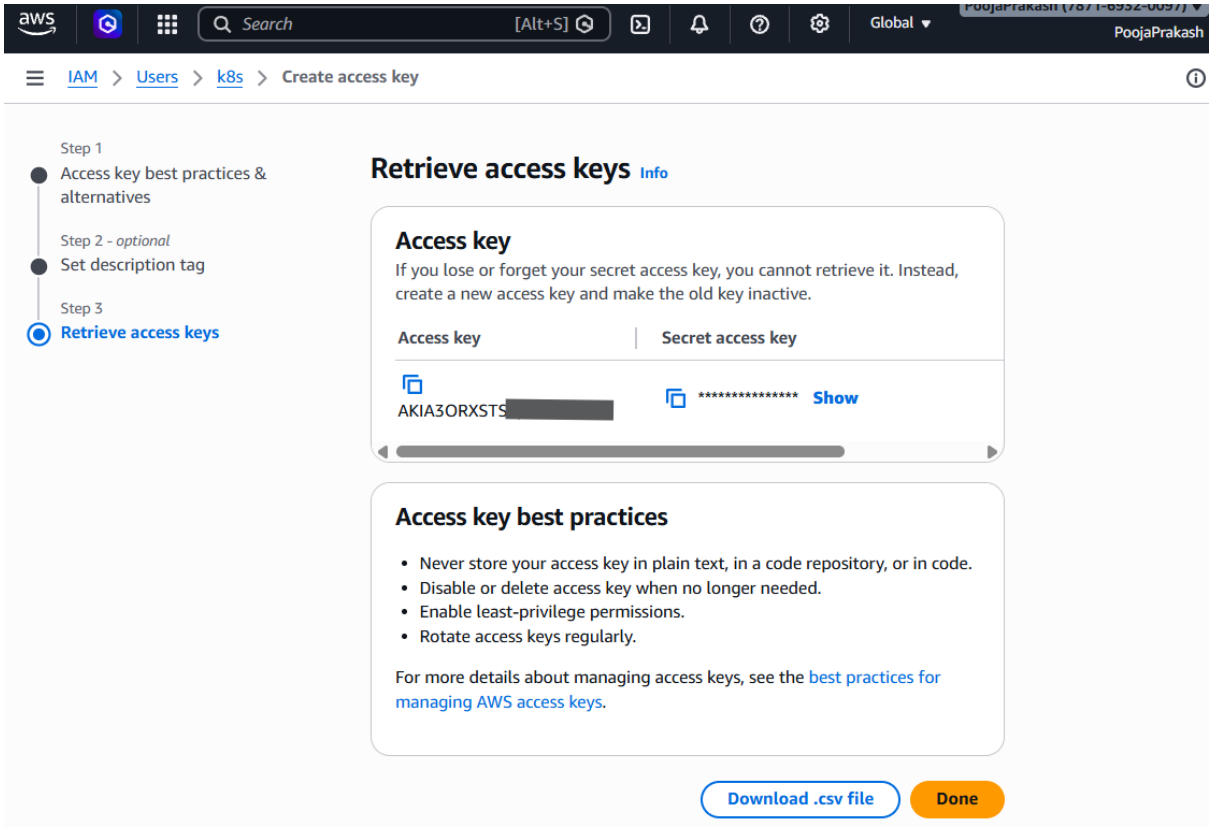

Dashboards › Import dashboard
Import dashboard
Import dashboard from file or Grafana.com
Importing dashboard from Grafana.com
Published by
haryan
Updated on
2023-08-24 15:04:53
Options
Name
Jenkins: Performance and Health Overview
Folder
Dashboards
Unique identifier (UID)
The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
haryan-jenkins
Change uid
DS_PROMETHEUS
Select a prometheus data source
prometheus
Import
Cancel

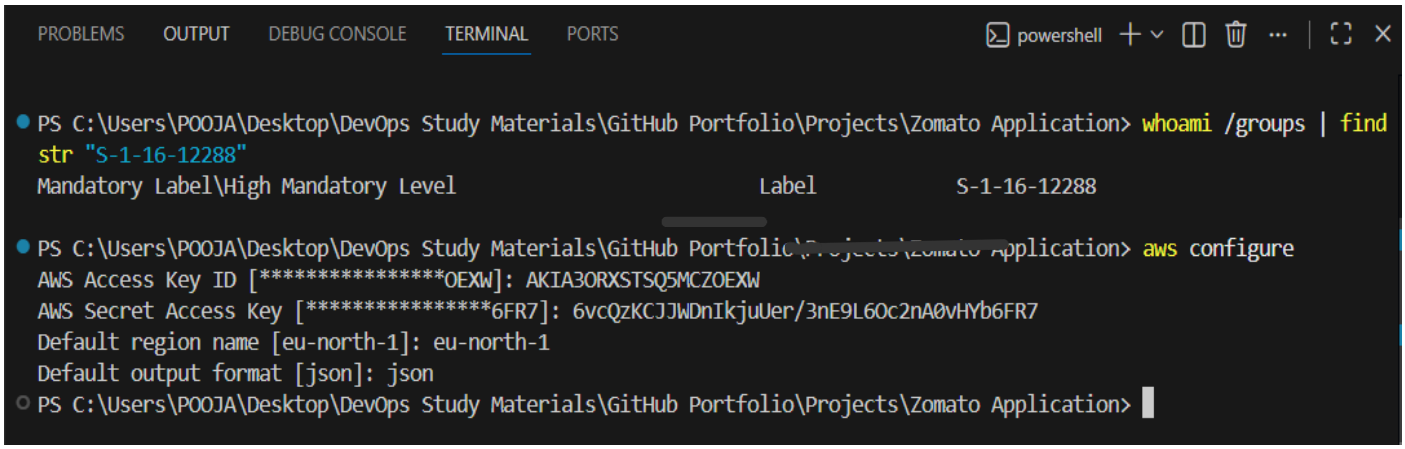

Not secure
13.60.245.32:3000/d/haryan-jenkins/jenkins3a-performance-and-health-overview?orgId=1&from=now-1h&to=now&timezone=browser
Dashboards › Jenkins: Performance and Health Overview
Search...
ctrl+k
Add
Settings
Exit edit
Save dashboard
Last 1 hour
Refresh
Uptime
5000000
4000000
3000000
2000000
1000000
22:50
23:00
23:10
23:20
23:30
23:40
{__name__="default_jenkins_uptime", instance="13.61.0.44:8080", job="jenki
default_jenkins_uptime {__name__="default_jenkins_uptime", instance="13.61
Total Build Count
3000000000
2000000000
1000000000
0
22:50
23:00
23:10
23:20
23:30
23:40
{__name__="default_jenkins_builds_total_build_count_created", buildable="tru
default_jenkins_builds_total_build_count_created {__name__="default_jenkins
Last Build Duration
2000000
1500000
1000000
500000
22:50
23:00
23:10
23:20
23:30
23:40
="default_jenkins_builds_last_build_duration_milliseconds", buildable="true", insta
kins_builds_last_build_duration_milliseconds {__name__="default_jenkins_builds_
Job Logs
2
1.5
1
0.5
0
22:50
23:00
23:10
23:20
23:30
23:40
{__name__="default_jenkins_builds_job_log_updated", buildable="true", instal
Jenkins Build Duration
3300000
3200000
3100000
3000000
2900000
2800000
22:50
23:00
23:10
23:20
23:30
23:40
13.61.0.44:8080", jenkins_job="Zomato-Application", job="jenkins", repo="NA"}
on_milliseconds_summary_sum", buildable="true", instance="13.61.0.44:8080", jer
Health Score
200
150
100
50
0
22:50
23:00
23:10
23:20
23:30
23:40
{__name__="default_jenkins_builds_health_score", buildable="true", instance
default_jenkins_builds_health_score {__name__="default_jenkins_builds_healt

Phase 3: Kubernetes Deployment using Amazon EKS

1. EKS Cluster Setup

- Generated AWS Access Key and Secret Key from IAM user and configured AWS CLI in VS Code.





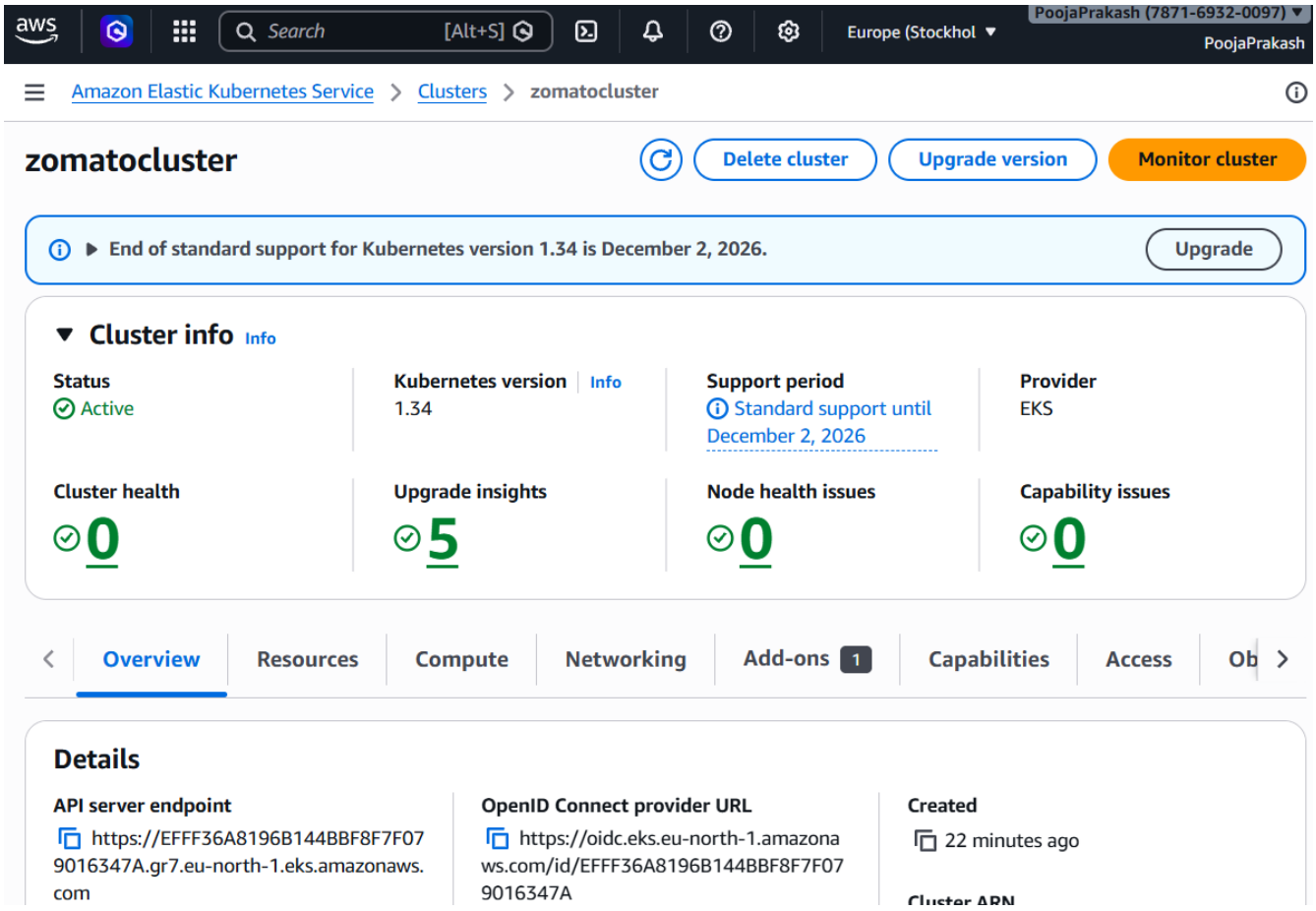
- Installed kubectl and eksctl

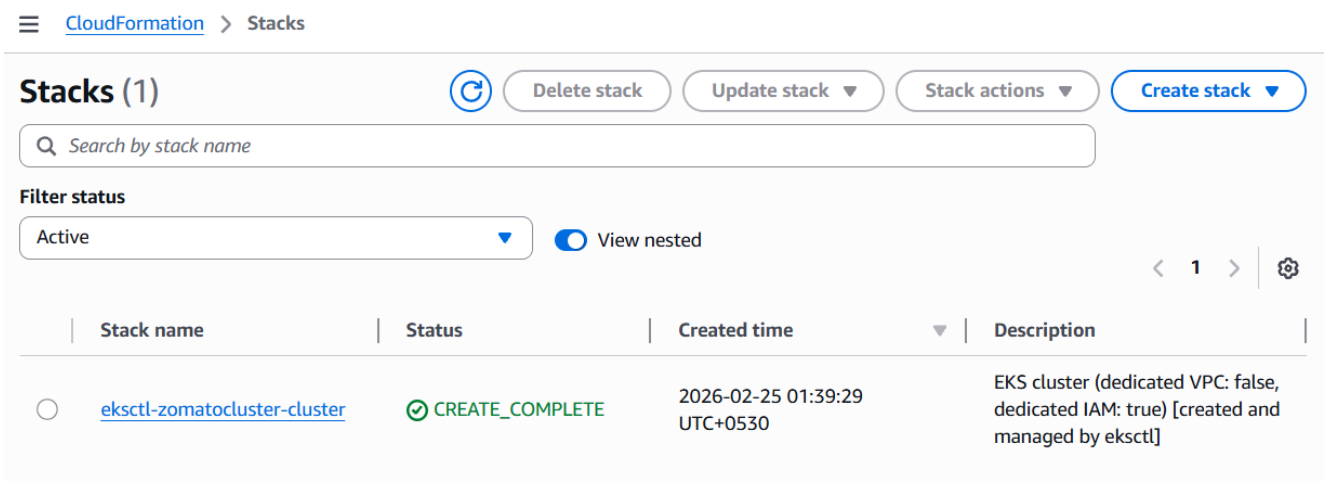
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                   powershell
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl version --client
Client Version: v1.28.2
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl version
0.223.0
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

- Created an EKS cluster named zomatocluster in the same region and VPC.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl create cluster --name=zom
atocluster --region=eu-north-1 --vpc-public-subnets=subnet-02c17cda1d166a85b,subnet-061485a8046389ab0 --without-nodegroup
```

- Verified cluster and CloudFormation stacks in AWS Console.





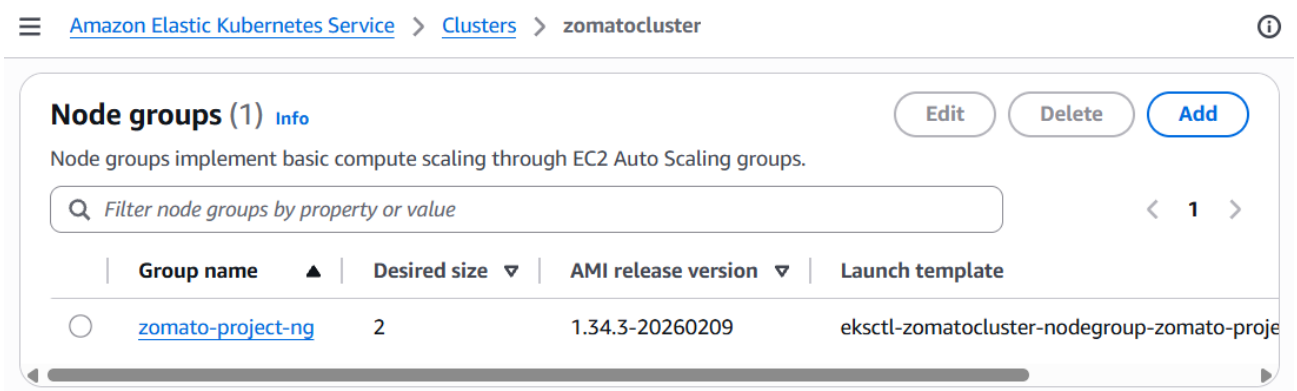
- Associated OIDC identity provider to enable IAM roles for service accounts.

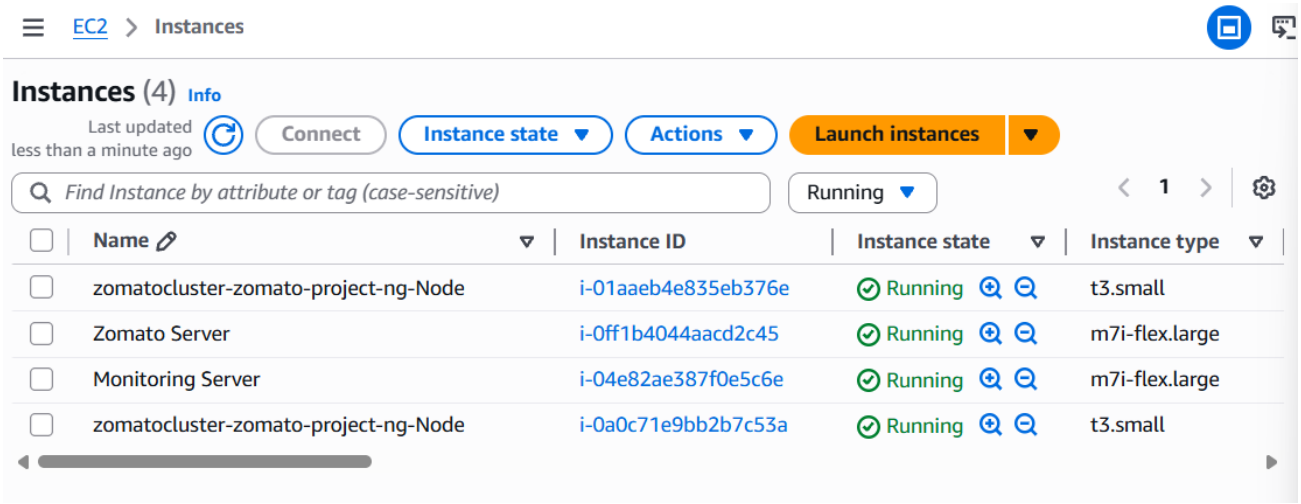
```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl utils associate-iam-oidc-provider --region eu-north-1 --cluster zomatocluster --approve
2026-02-25 02:17:01 [ℹ]  will create IAM Open ID Connect provider for cluster "zomatocluster" in "eu-north-1"
2026-02-25 02:17:03 [✓]  created IAM Open ID Connect provider for cluster "zomatocluster" in "eu-north-1"
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

- Created a Node Group in public subnets with required add-ons.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl create nodegroup --cluster=zomatocluster --region=eu-north-1 --name=zomato-project-ng --node-type=t3.small --nodes=2 --nodes-min=2 --nodes-max=4 --node-volume-size=20 --ssh-access --ssh-public-key=linux1 --managed --asg-access --external-dns-access --full-ecr-access --appmesh-access --alb-ingress-access
```

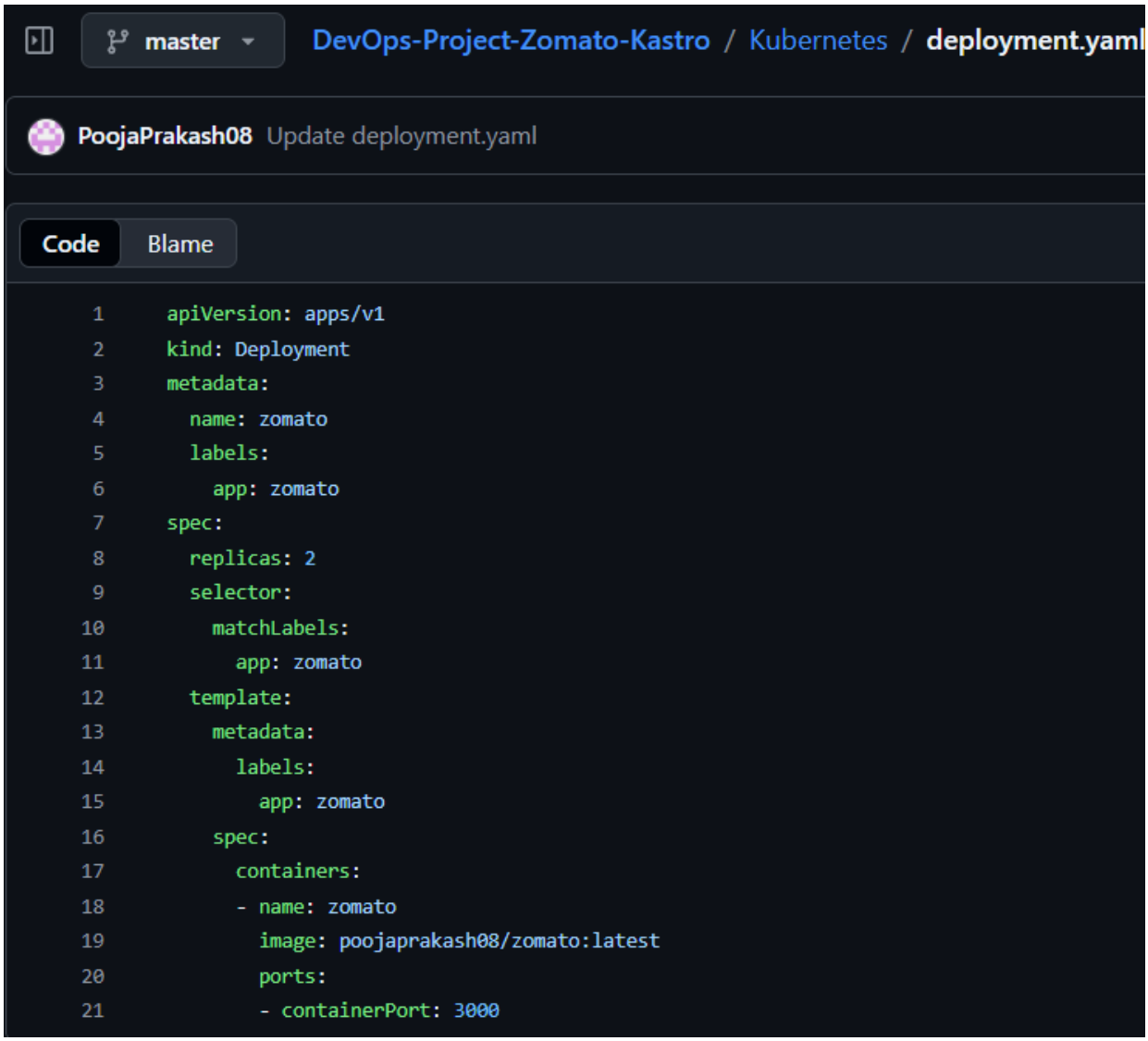
- Verified node group and worker nodes.





2. Kubernetes Deployment

- Created Kubernetes YAML files for deployment and service.



master DevOps-Project-Zomato-Kastro / Kubernetes / node-service.yaml

KastroVKiran Add files via upload

Code Blame

```yaml
kind: Service
apiVersion: v1
metadata:
  name: node-exporter
  namespace: prometheus-node-exporter
spec:
  selector:
      app: node-exporter
  ports:
  - name: node-exporter
    protocol: TCP
    port: 9100
    targetPort: 9100
  type: NodePort
```

master DevOps-Project-Zomato-Kastro / Kubernetes / service.yaml

KastroVKiran Update service.yaml

Code Blame

```yaml
apiVersion: v1
kind: Service
metadata:
  name: zomato
  labels:
    app: zomato
spec:
  type: NodePort
  ports:
  - port: 3000
    targetPort: 3000
    nodePort: 30001
  selector:
    app: zomato
```

- Installed Argo CD for GitOps-based deployment.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl create namespace argocd
namespace/argocd created
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/v2.4.7/manifests/install.yaml
```

- Exposed Argo CD server publicly.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl edit svc argocd-server -n argocd
service/argocd-server edited
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get svc argocd-server -n argocd
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP
  PORT(S)                      AGE
argocd-server   LoadBalancer   172.20.22.120   a1530b493f36c4d2289a6791c45fd47c-1170830466.eu-north-1.elb.amazonaws.com
  80:30966/TCP,443:32133/TCP   8m6s
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get ns
NAME              STATUS   AGE
argocd            Active   12m
default           Active   67m
kube-node-lease   Active   67m
kube-public       Active   67m
kube-system       Active   67m
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          11m
argocd-applicationset-controller-696ccc6458-ccwmk   1/1     Running   0          11m
argocd-dex-server-7868949c59-h6jvf                  1/1     Running   0          11m
argocd-notifications-controller-65b84457bb-4fntd    1/1     Running   0          11m
argocd-redis-868cb656f8-2jgts                       1/1     Running   0          11m
argocd-repo-server-6c6b5c6c78-mspht                 1/1     Running   0          11m
argocd-server-9c88f7cf9-bw6lc                       1/1     Running   0          11m
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

- Installed Node Exporter using Helm for Kubernetes monitoring.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> choco --version
2.6.0
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> choco install kubernetes-hel
m -y
Chocolatey v2.6.0
Installing the following packages:
kubernetes-helm
```

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> helm version
version.BuildInfo{Version:"v4.1.0", GitCommit:"4553a0a96e5205595079b6757236cc6f969ed1b9", GitTreeState:"clean", GoVersion:"
go1.25.6", KubeClientVersion:"v1.35"}
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> helm repo add prometheus-com
munity https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl create namespace pro
metheus-node-exporter
namespace/prometheus-node-exporter created
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> helm install prometheus-node
-exporter prometheus-community/prometheus-node-exporter --namespace prometheus-node-exporter
NAME: prometheus-node-exporter
LAST DEPLOYED: Wed Feb 25 03:00:19 2026
NAMESPACE: prometheus-node-exporter
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
1. Get the application URL by running these commands:
  export POD_NAME=$(kubectl get pods --namespace prometheus-node-exporter -l "app.kubernetes.io/name=prometheus-node-export
er,app.kubernetes.io/instance=prometheus-node-exporter" -o jsonpath="{.items[0].metadata.name}")
  echo "Visit http://127.0.0.1:9100 to use your application"
  kubectl port-forward --namespace prometheus-node-exporter $POD_NAME 9100
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

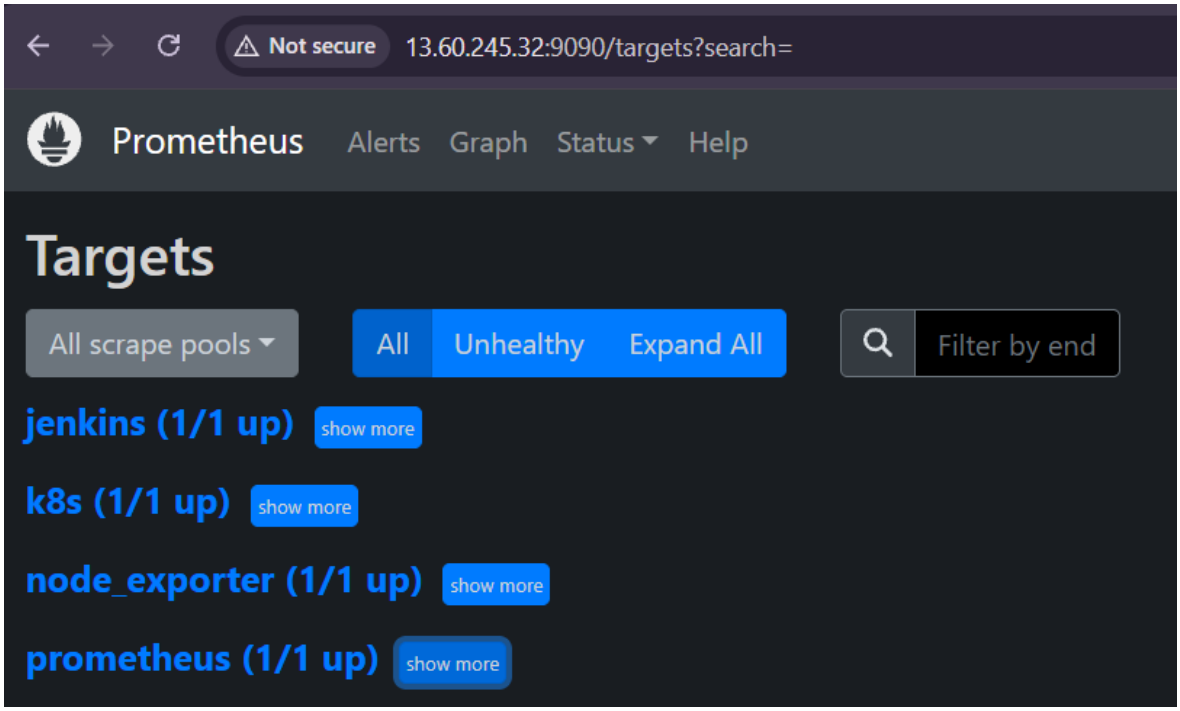
```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get pods -n kube-sys
tem
NAME                              READY   STATUS    RESTARTS   AGE
aws-node-7jp4q                    2/2     Running   0          43m
aws-node-9m2z9                    2/2     Running   0          43m
coredns-65cf8c5645-8np9t          1/1     Running   0          84m
coredns-65cf8c5645-w297z          1/1     Running   0          84m
kube-proxy-6bts4                  1/1     Running   0          43m
kube-proxy-h4t7h                  1/1     Running   0          43m
metrics-server-b59777598-8dvxv    1/1     Running   0          82m
metrics-server-b59777598-f8v9j    1/1     Running   0          82m
```

- Updated prometheus.yml to scrape Kubernetes metrics.

```
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['13.60.245.32:9100']

  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    static_configs:
      - targets: ['13.61.0.44:8080']

  - job_name: 'k8s'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['13.53.235.166:9100']
-- INSERT --                                                    43,40
```

```
root@ip-10-0-6-39:~# sudo vi /etc/prometheus/prometheus.yml
root@ip-10-0-6-39:~# promtool check config /etc/prometheus/prometheus.yml
Checking /etc/prometheus/prometheus.yml
 SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax

root@ip-10-0-6-39:~#
```

- Verified all targets were up in Prometheus.

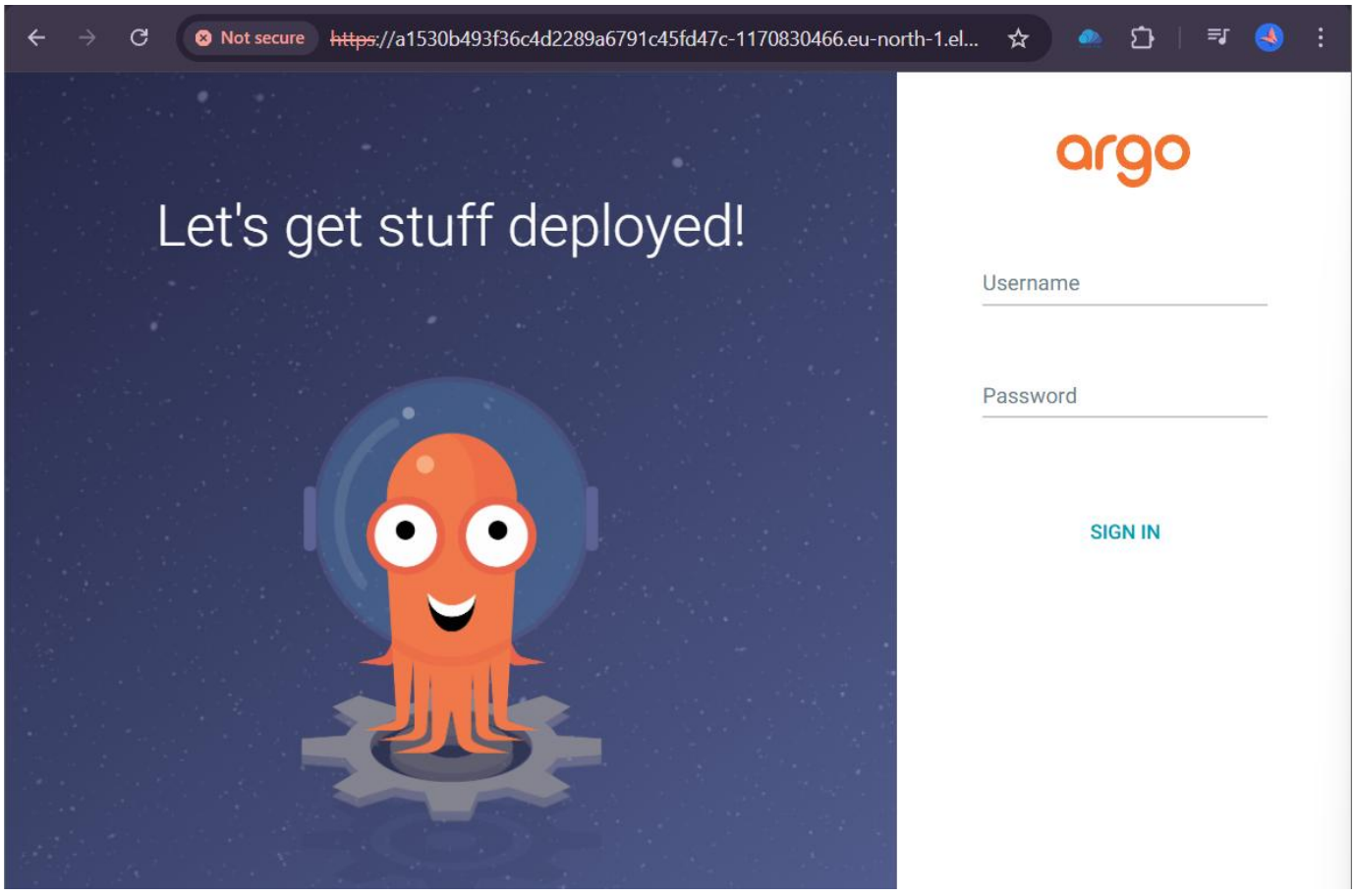


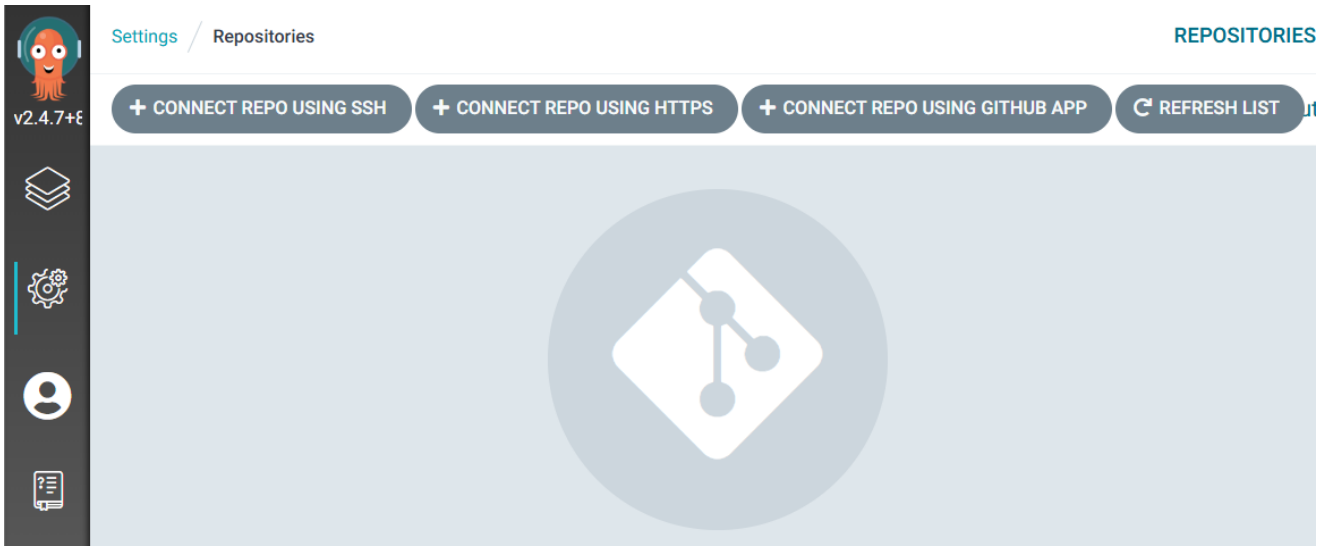
3. Argo CD Application Deployment

- Accessed Argo CD using Load Balancer URL and logged in.

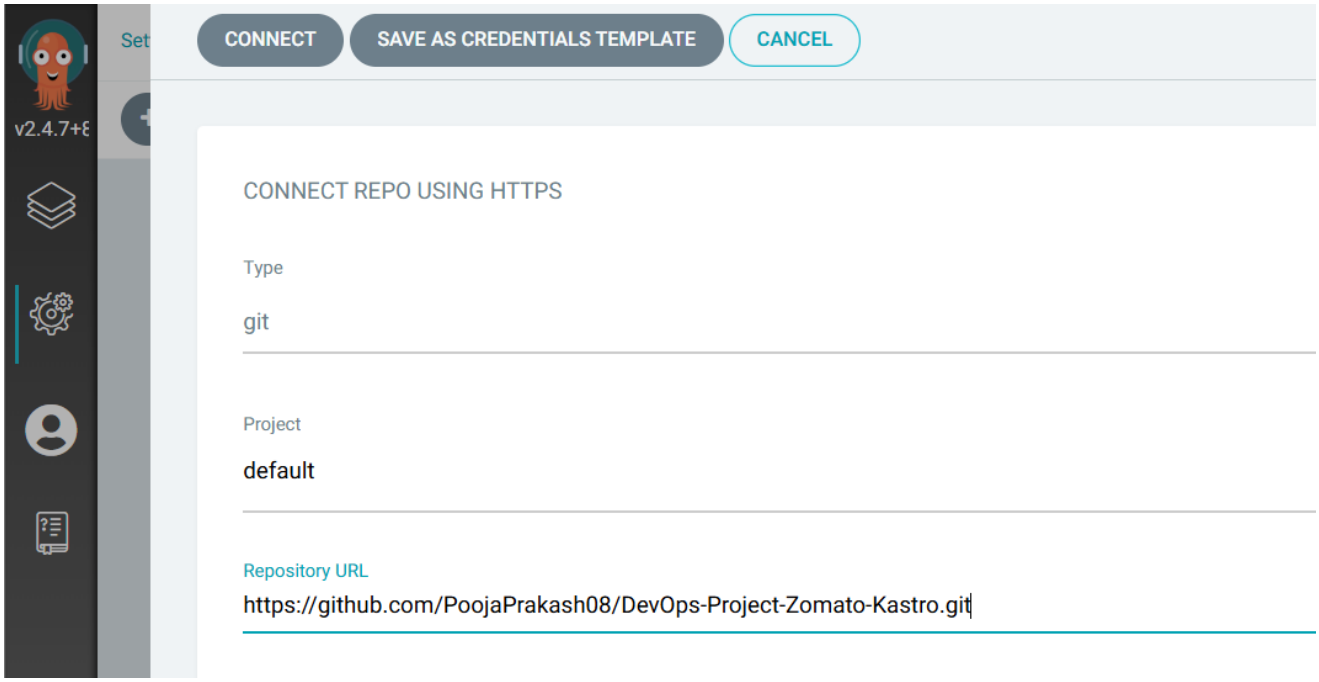
```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> $ARGOCD_SERVER = (kubectl get svc argocd-server -n argocd -o json | ConvertFrom-Json).status.loadBalancer.ingress[0].hostname
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> $ARGOCD_SERVER
a1530b493f36c4d2289a6791c45fd47c-1170830466.eu-north-1.elb.amazonaws.com
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

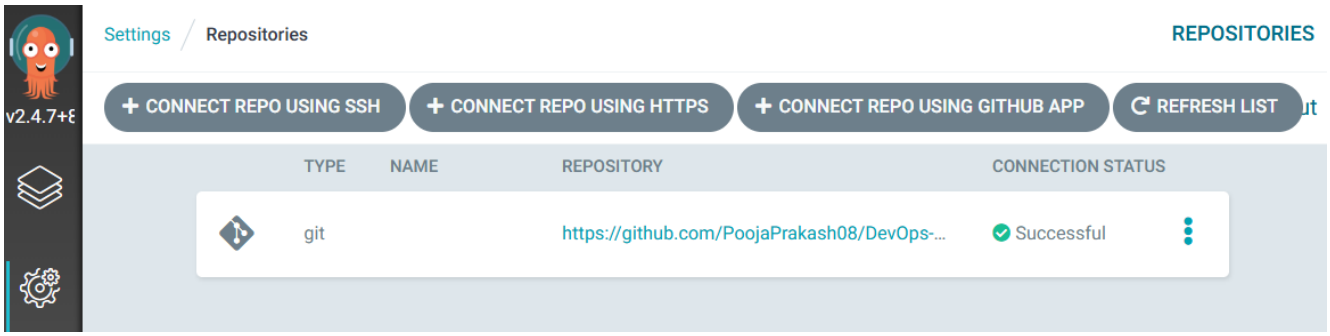
```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> [System.Text.Encoding]::UTF8
.GetString(
>>   [System.Convert]::FromBase64String(
>>     (kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}")
>>   )
>> )
QrOVNAH-Ghc7QjbY
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```



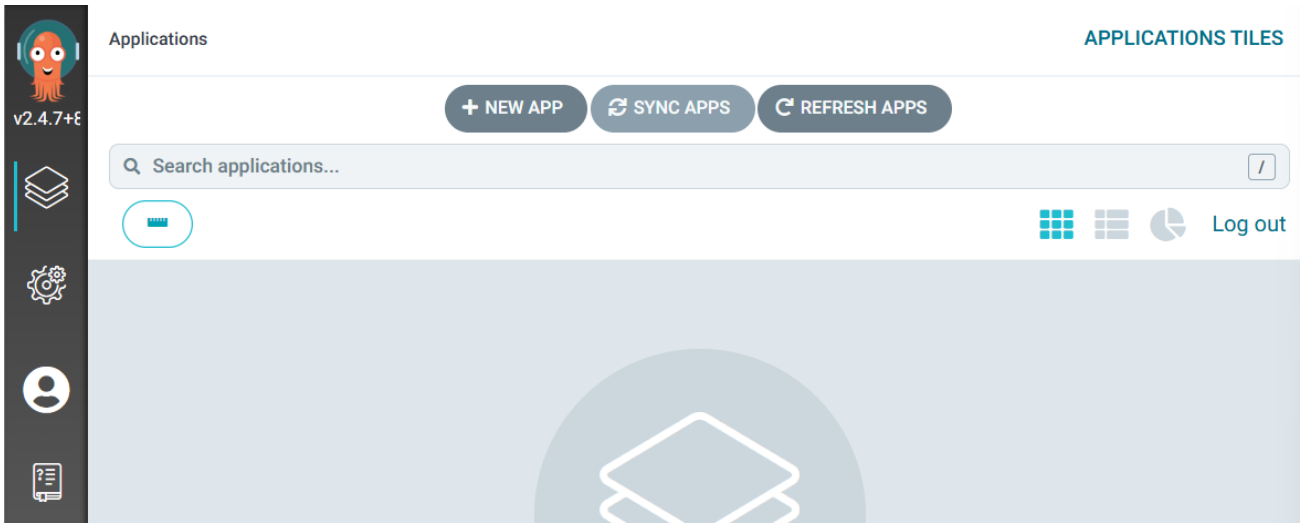


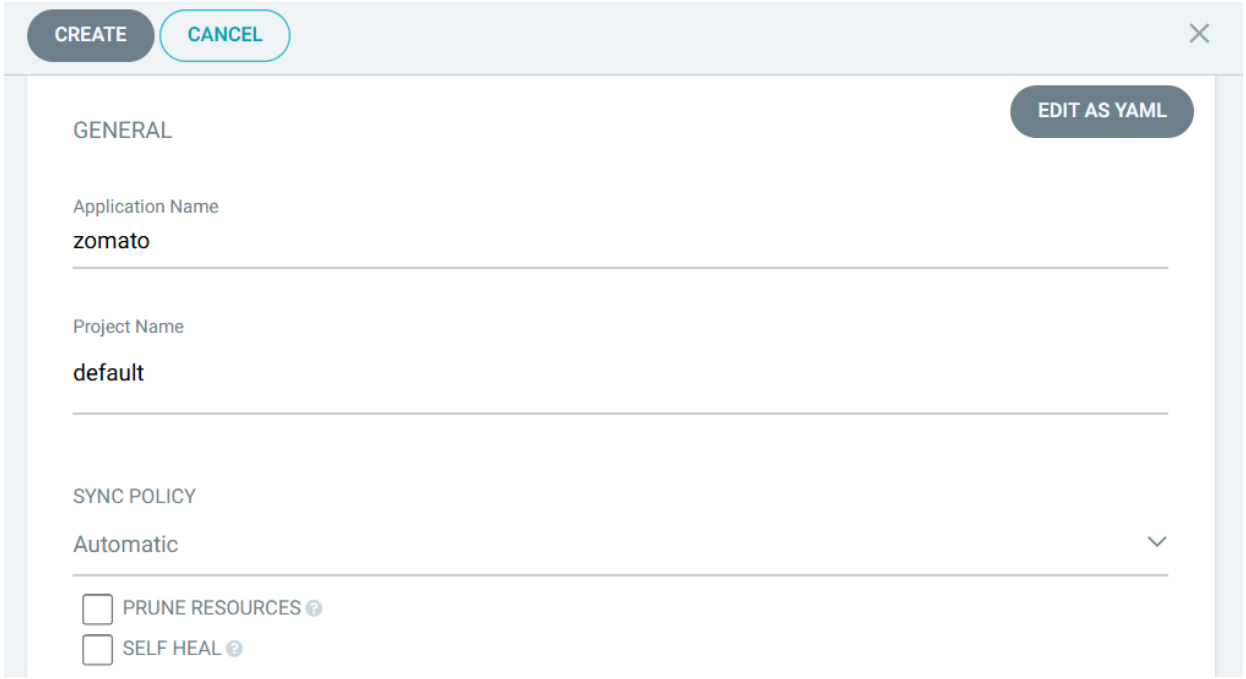
- Connect to the Zomato application repository in Argo CD.

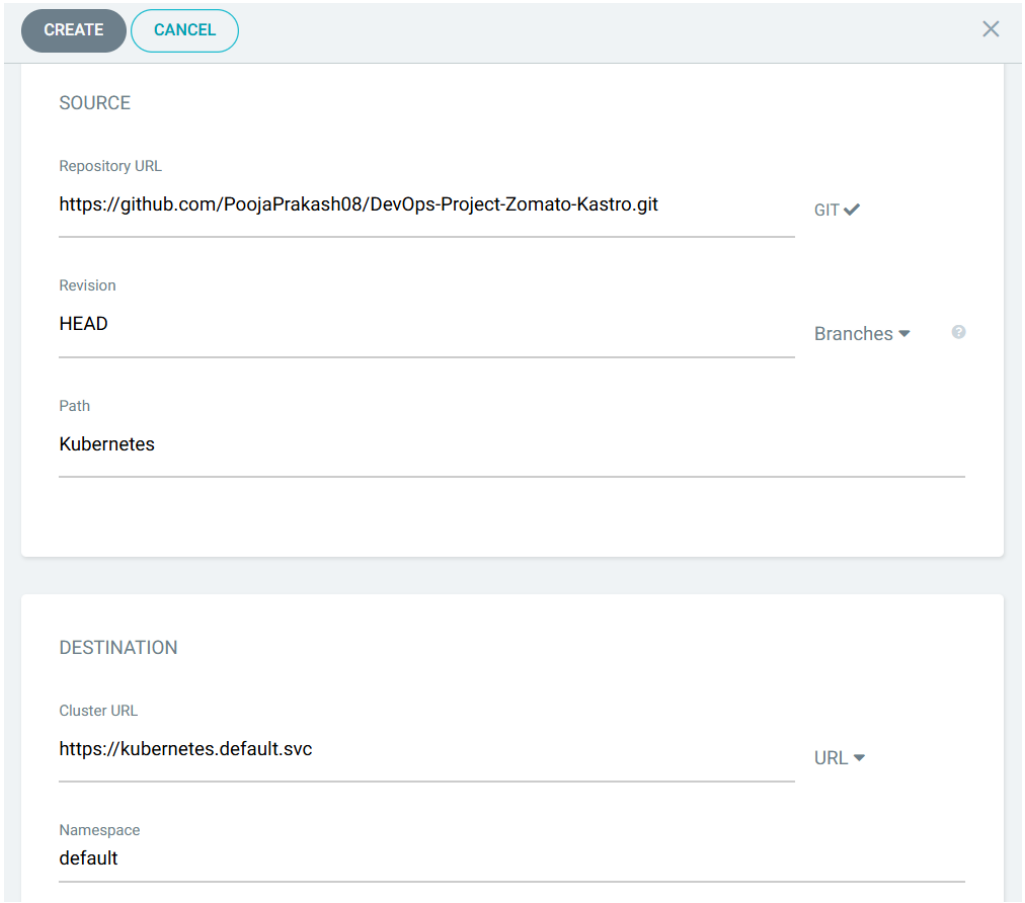


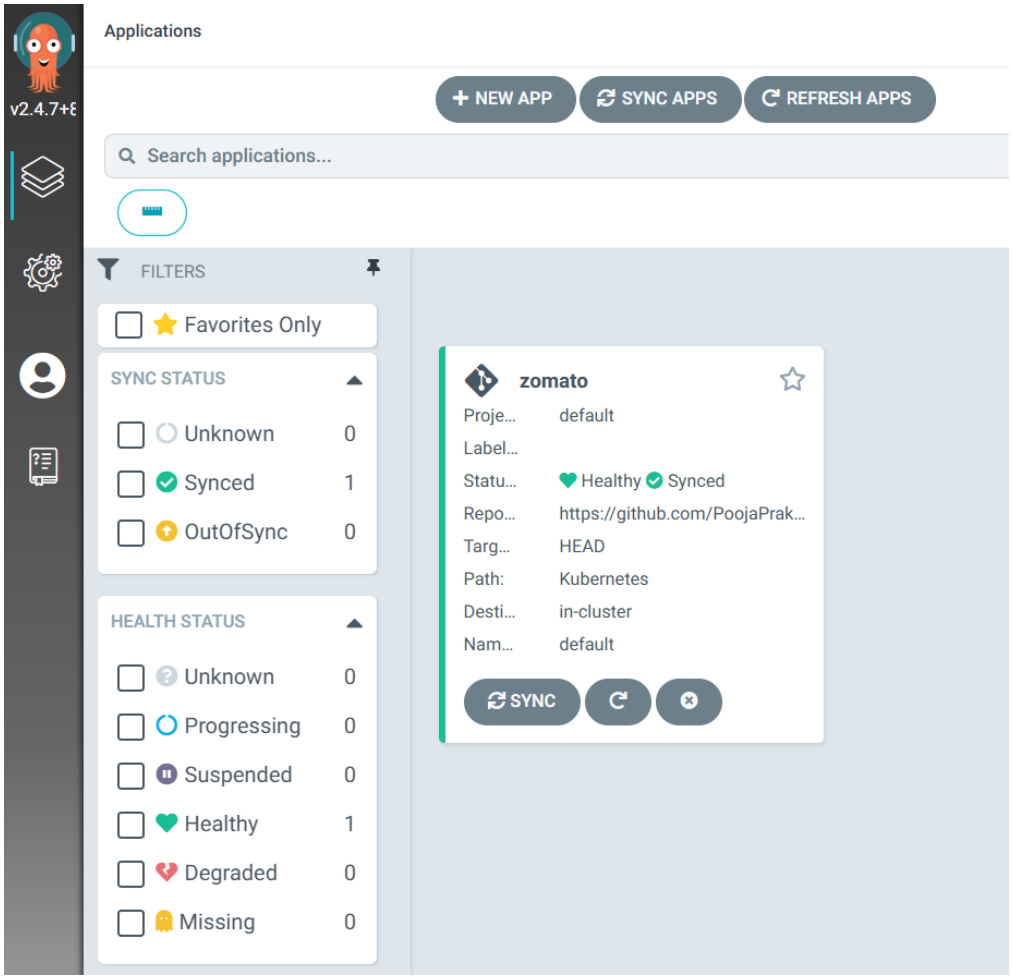


- Add the Zomato application in the argocd

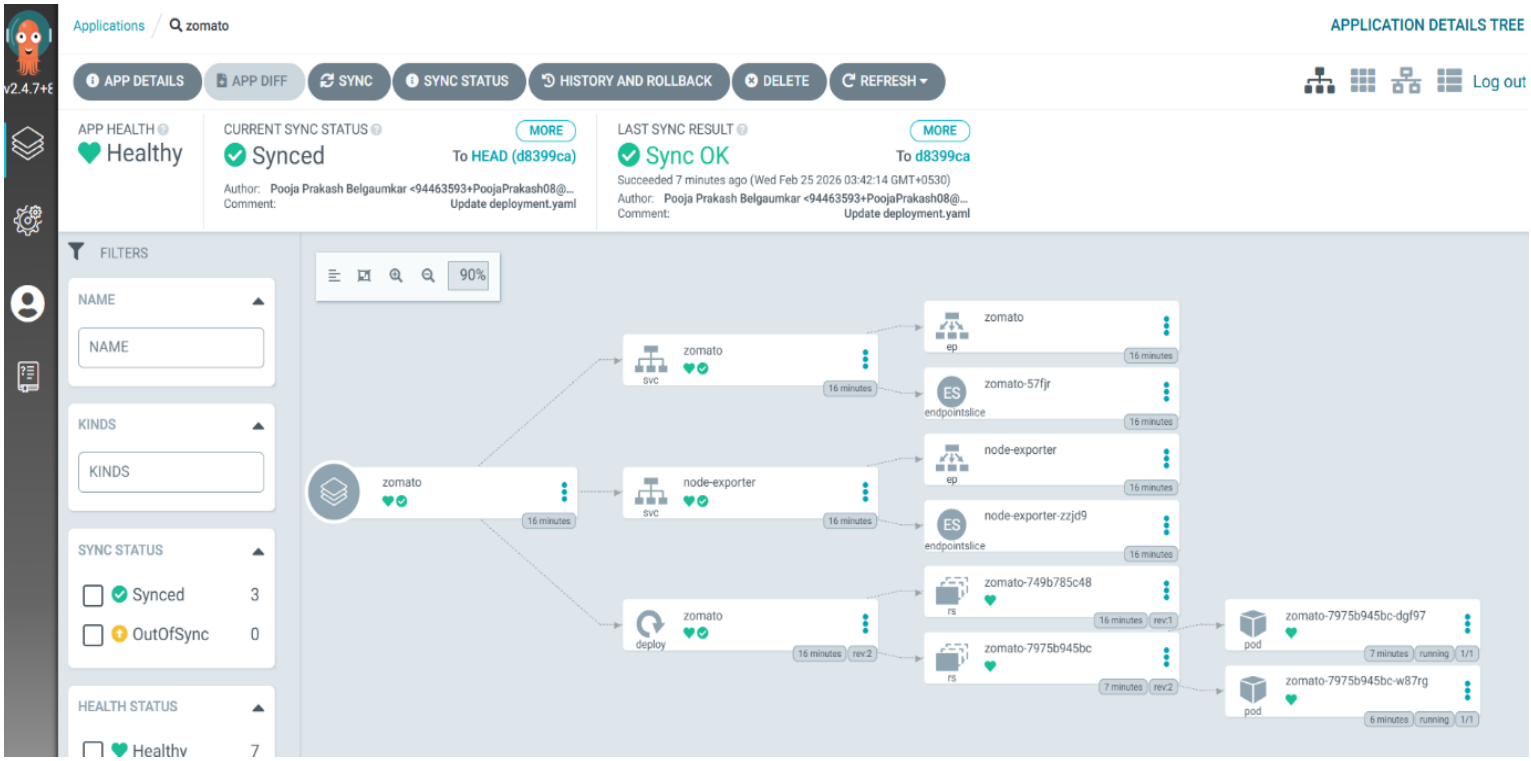




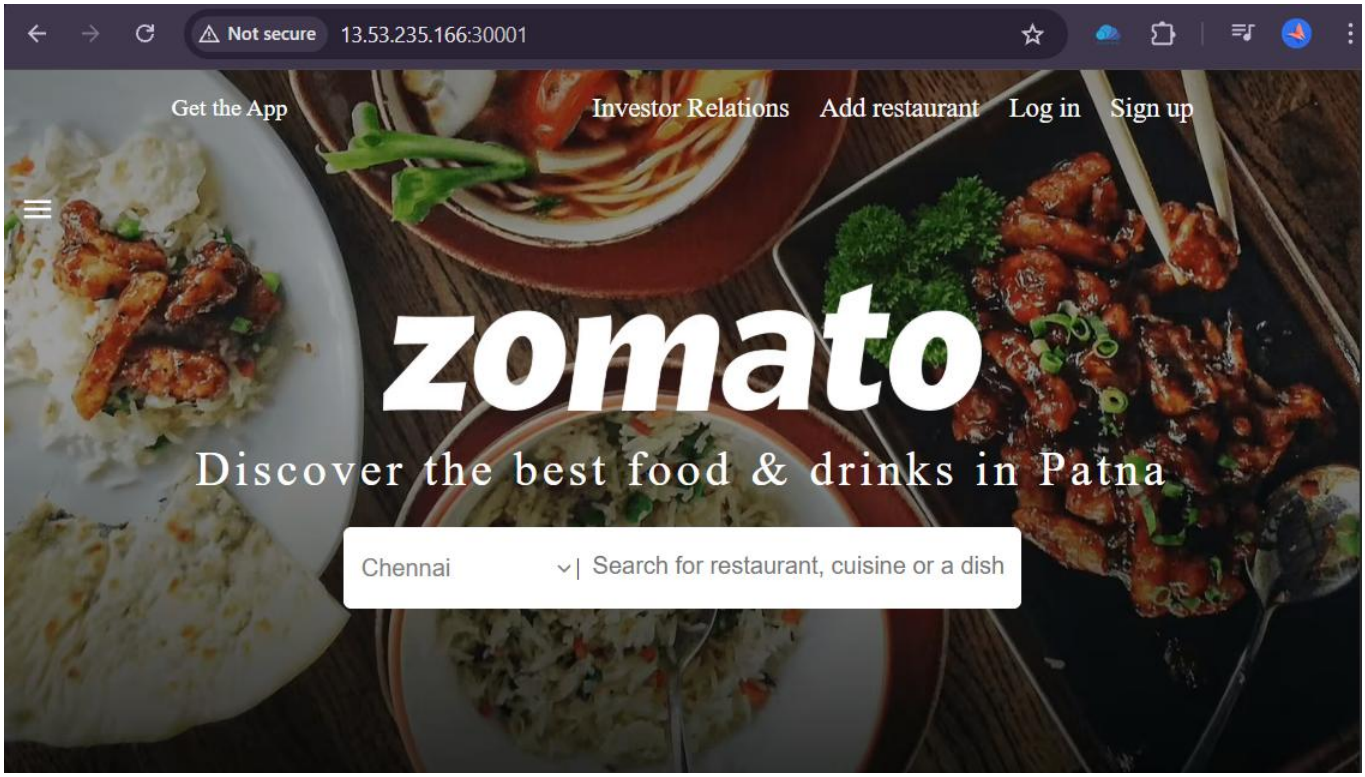

CREATE
CANCEL
SOURCE
Repository URL
https://github.com/PoojaPrakash08/DevOps-Project-Zomato-Kastro.git
GIT
Revision
HEAD
Branches
Path
Kubernetes
DESTINATION
Cluster URL
https://kubernetes.default.svc
URL
Namespace
default


Applications
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
FILTERS
Favorites Only
SYNC STATUS
Unknown 0
Synced 1
OutOfSync 0
HEALTH STATUS
Unknown 0
Progressing 0
Suspended 0
Healthy 1
Degraded 0
Missing 0
zomato
Proje... default
Label...
Statu... Healthy Synced
Repo... https://github.com/PoojaPrak...
Targ... HEAD
Path: Kubernetes
Desti... in-cluster
Nam... default
SYNC

- Monitored application health and sync status from Argo CD dashboard.



- Accessed the Kubernetes-deployed Zomato application using the Node public IP and NodePort (30001).



- ✓ This project demonstrates a complete end-to-end DevOps implementation covering CI/CD automation, containerization, security scanning, Kubernetes orchestration, GitOps deployment and real-time monitoring. By integrating Jenkins, SonarQube, Docker, Trivy, Prometheus, Grafana, Amazon EKS and Argo CD, the application lifecycle was fully automated from source code to production deployment. This implementation reflects modern DevOps best practices and provides a scalable, secure and observable cloud-native deployment architecture.